

Neural Networks in NLP

CS 485, Spring 2026

Applications of Natural Language Processing

Katrin Erk

College of Information and Computer Sciences
University of Massachusetts Amherst

[Slides from Mohit Iyyer, Richard Socher, Brendan O'Connor]

Bringing things together

- Logistic regression models as small neural networks
 - How do we build larger ones?
- Language models learn to recognize and mimic natural-sounding word sequences
- Word embeddings capture facets of word meaning and word use
 - Can we compose word embeddings into embeddings for larger phrases?
 - *Key idea: automatically determine how to combine embedding from different words*
- How do we built neural networks that are language models, and that use embeddings?

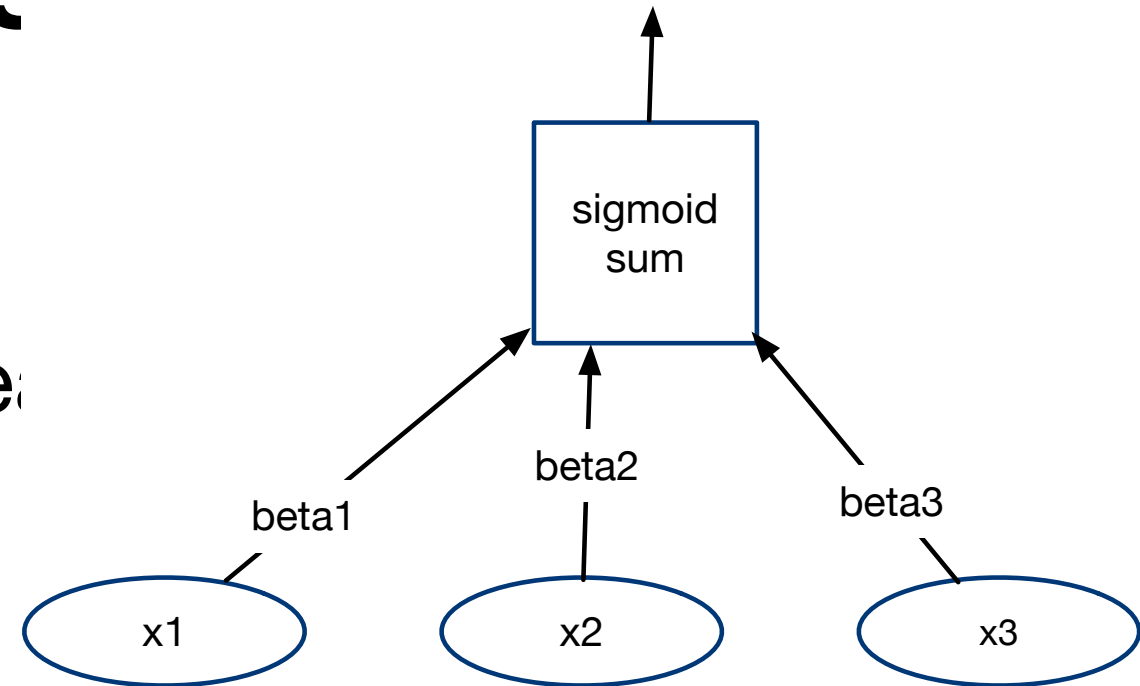
Neural Network models

- We've looked at linear models within logistic regression (for classification & language modeling)
- A proper neural network adds a **hidden layer** with a **non-linear activation function**, which allows (in theory) it to learn arbitrary functions (!) via yet more latent representations
- Pros:
 - flexible learning algorithm allows for customized architectures to reflect sequential language data
 - and thus latent representations for tokens, phrases, etc., beyond just word embeddings
 - and can make them really big, if you have lots of data
 - and can be parallelized internally (via GPUs or similar)
- Cons:
 - often unclear what it's learning internally ("black box")
 - they can get slow & compute-intensive

Reminder: logistic regression model

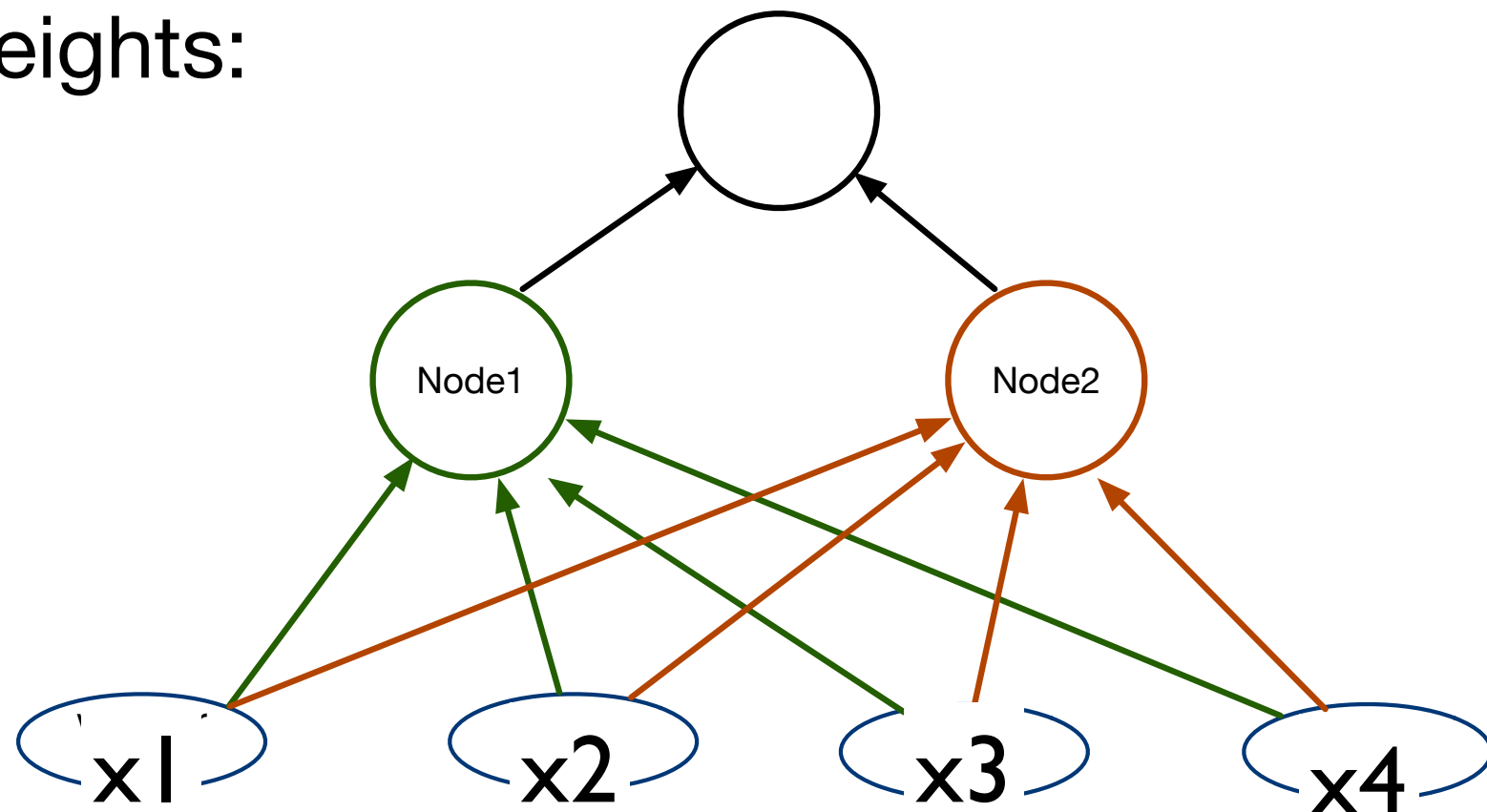
- Input datapoint $\mathbf{x}$: sequence of features
- Weights $\beta_1, \dots, \beta_n$
- Prediction

$$\hat{y} = \sigma(\beta_1 x_1 + \dots + \beta_n x_n) = \sigma(\beta \mathbf{x})$$



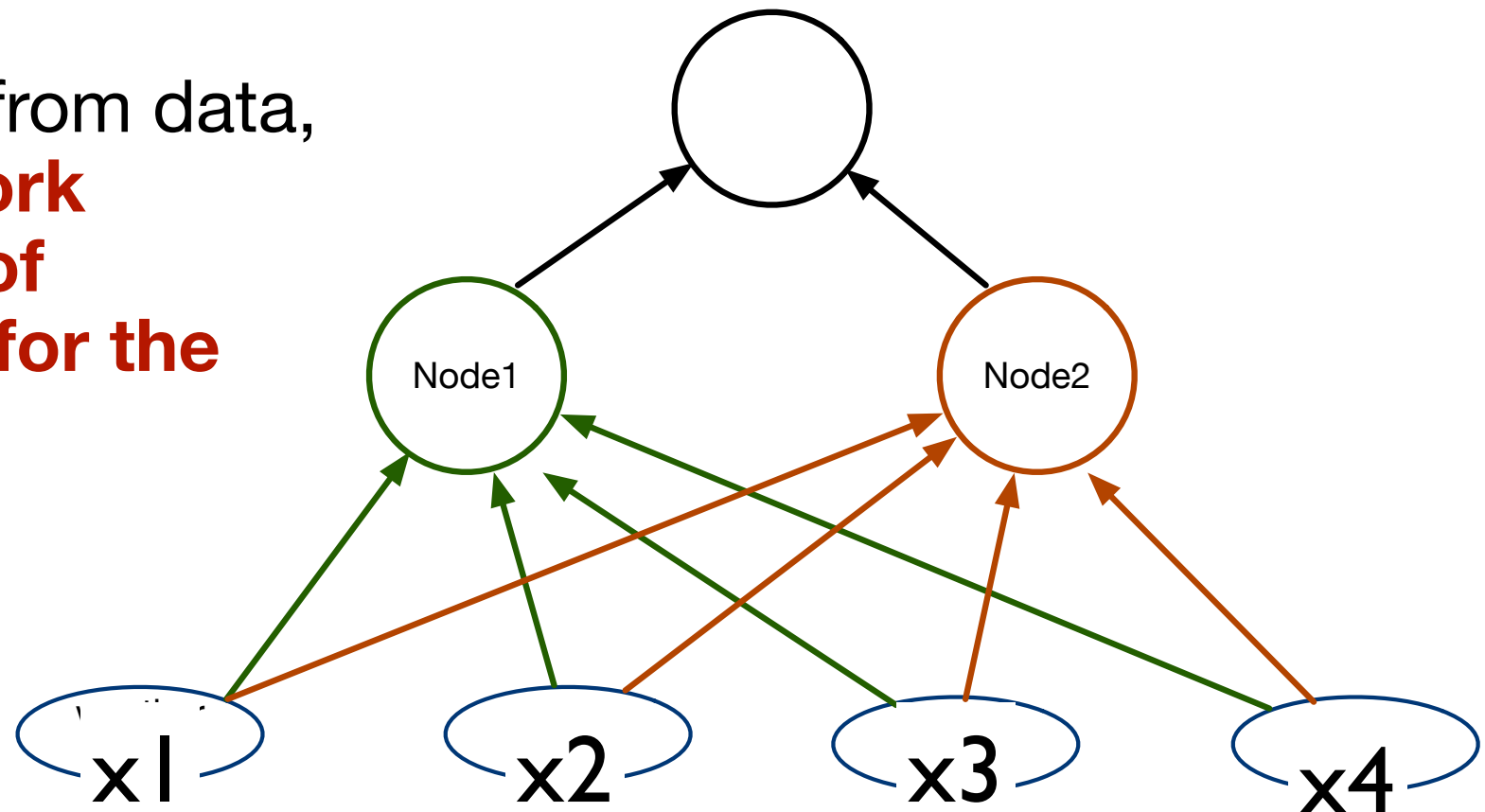
We can build more complex structures

- “submode” with green weights:
high for x1 and x3, low for x2, x4.
- “submode” with red weights:
high for x1, x2, x4



We can build more complex structures

- Node1, node2 are **combinations of input features**
- Output depends on these combined features
- If we learn all the weights from data, that means that **the network can learn combinations of input features as is best for the task: It can learn its own features!**



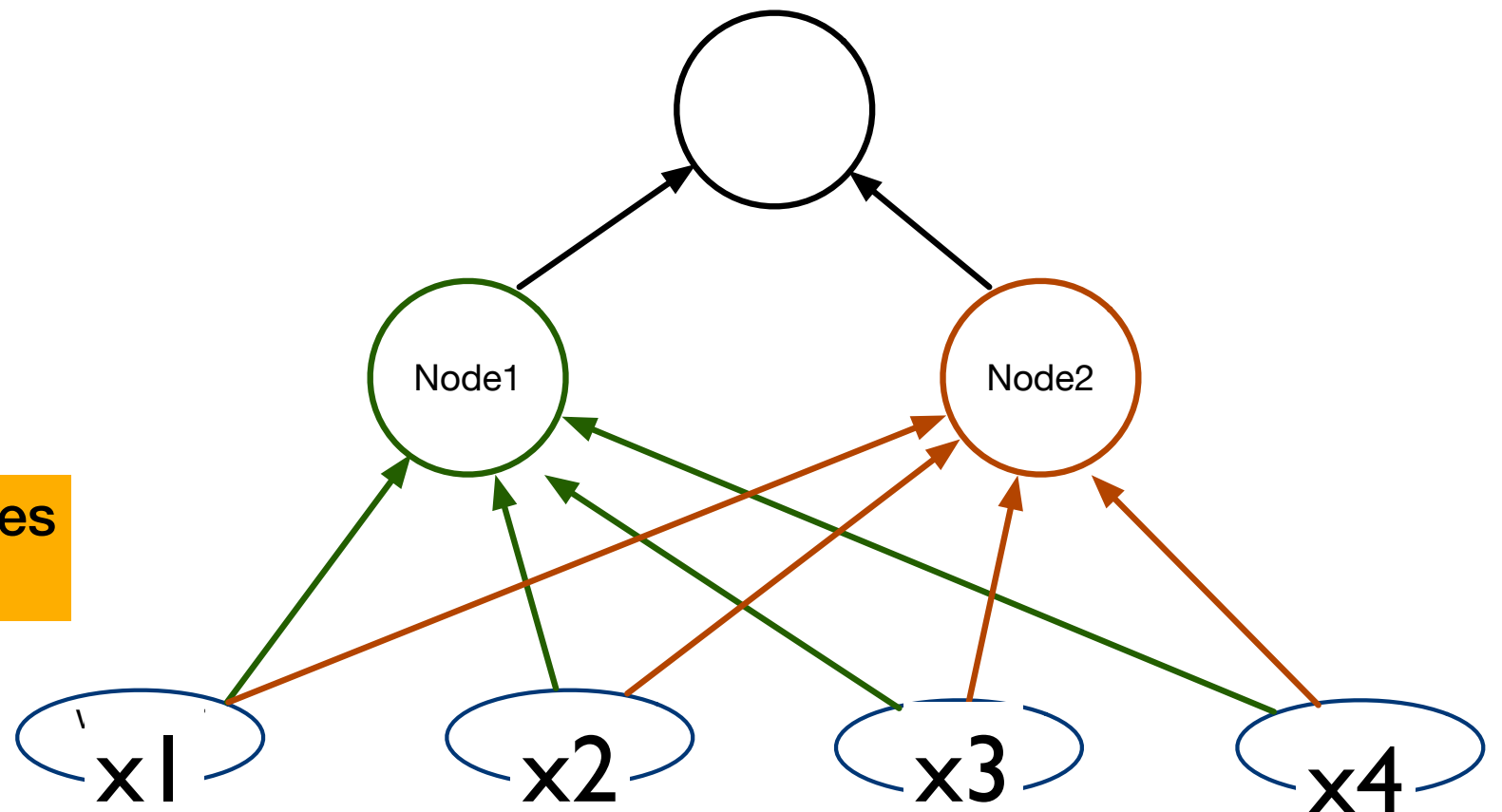
This is a multi-layer network

Output layer

Hidden layer

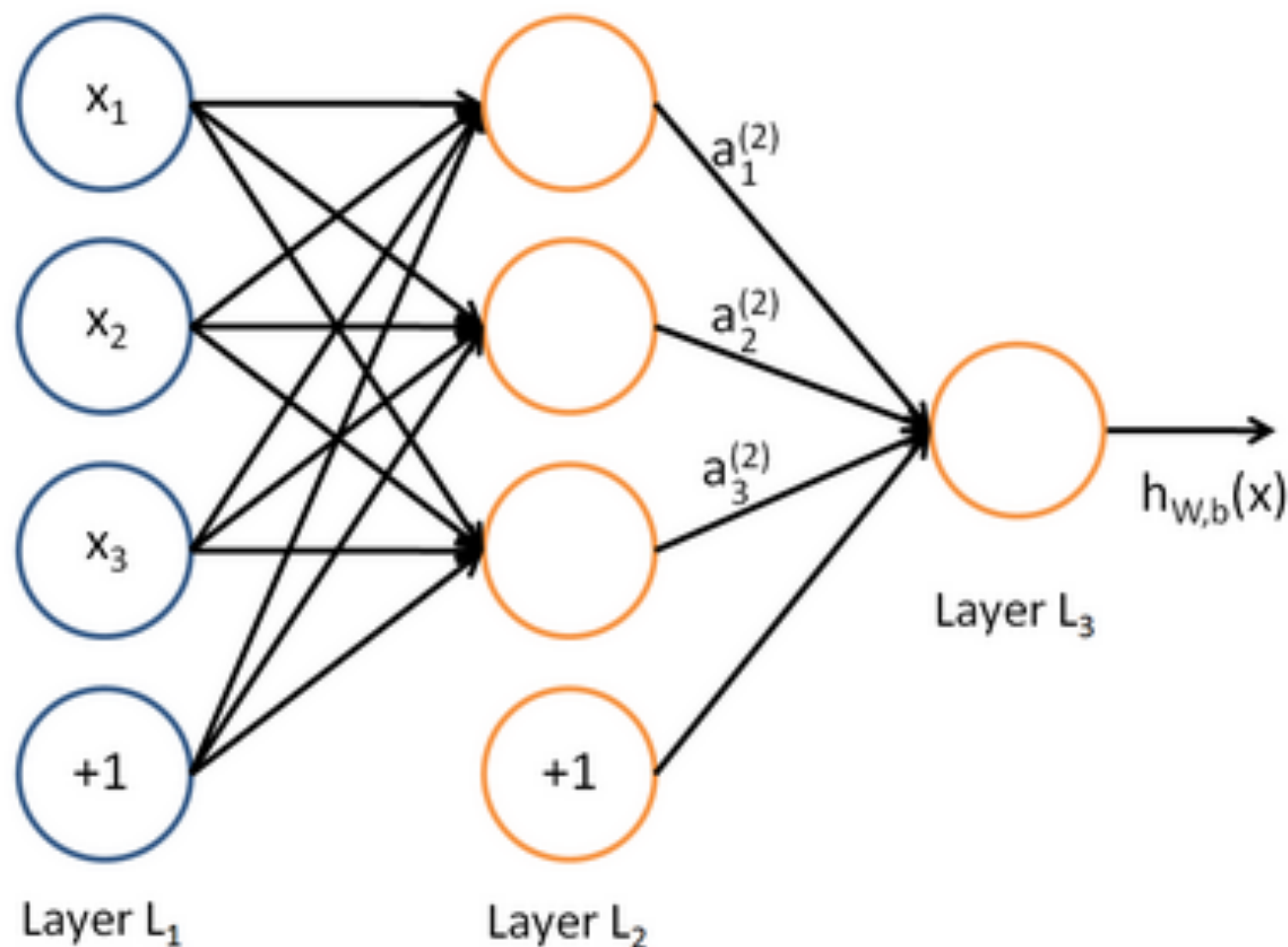
We can have more “hidden” nodes than just those two

Input layer



NN: kind of like several intermediate logregs

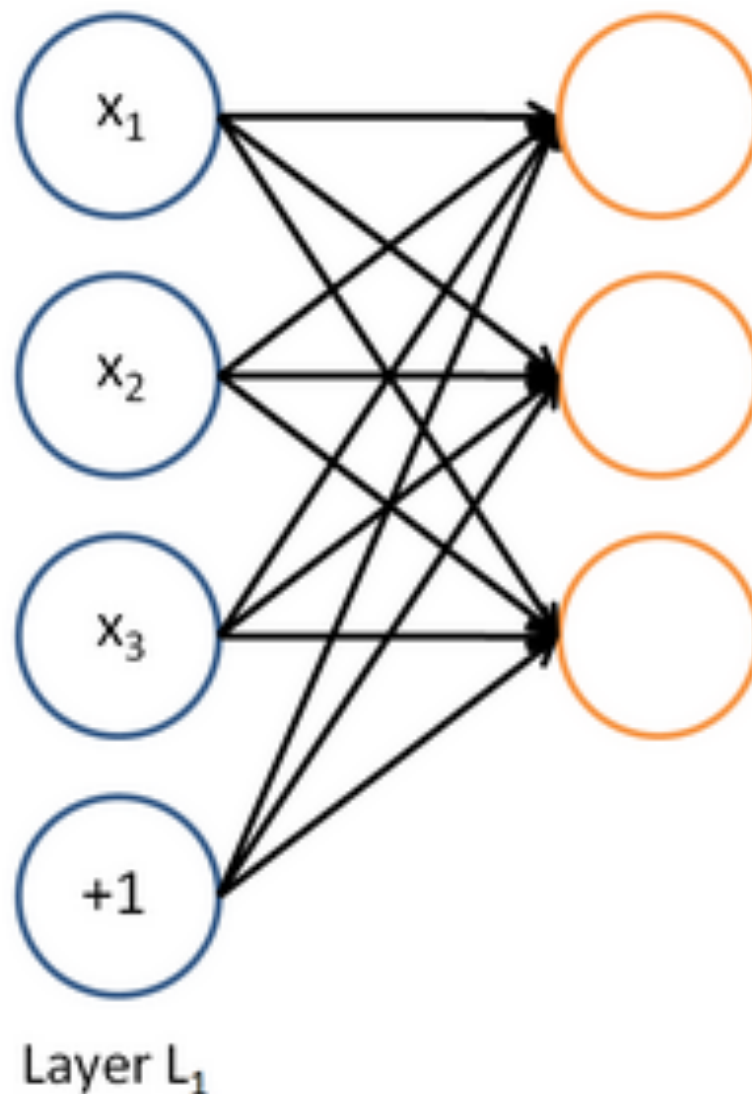
... which we can feed into another logistic regression function



It is the loss function that will direct what the intermediate hidden variables should be, so as to do a good job at predicting the targets for the next layer, etc.

NN: like several intermediate logregs

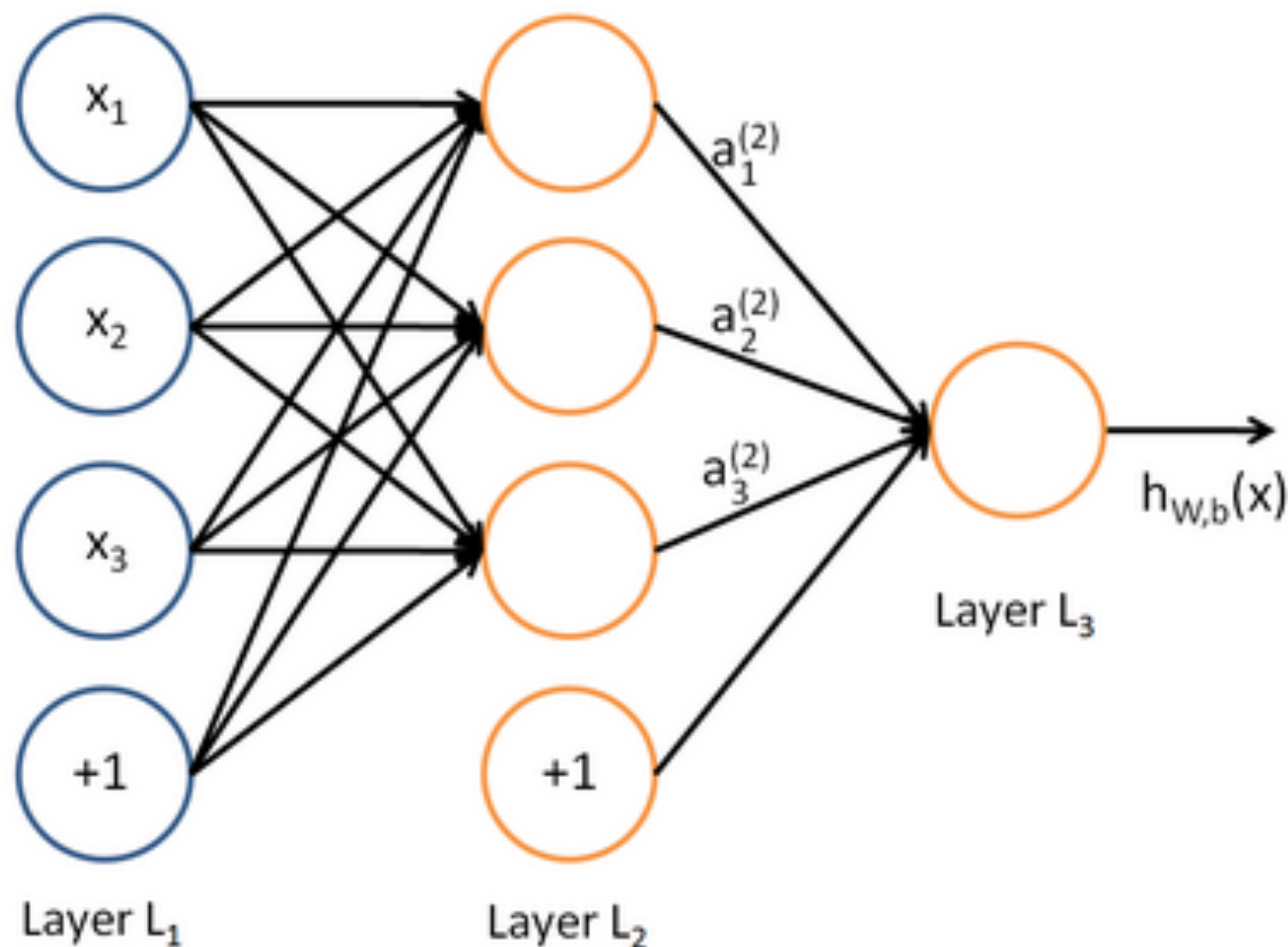
If we feed a vector of inputs through a bunch of logistic regression functions, then we get a vector of outputs ...



But we don't have to decide ahead of time what variables these logistic regressions are trying to predict!

NN: like several intermediate logregs

... which we can feed into another logistic regression function

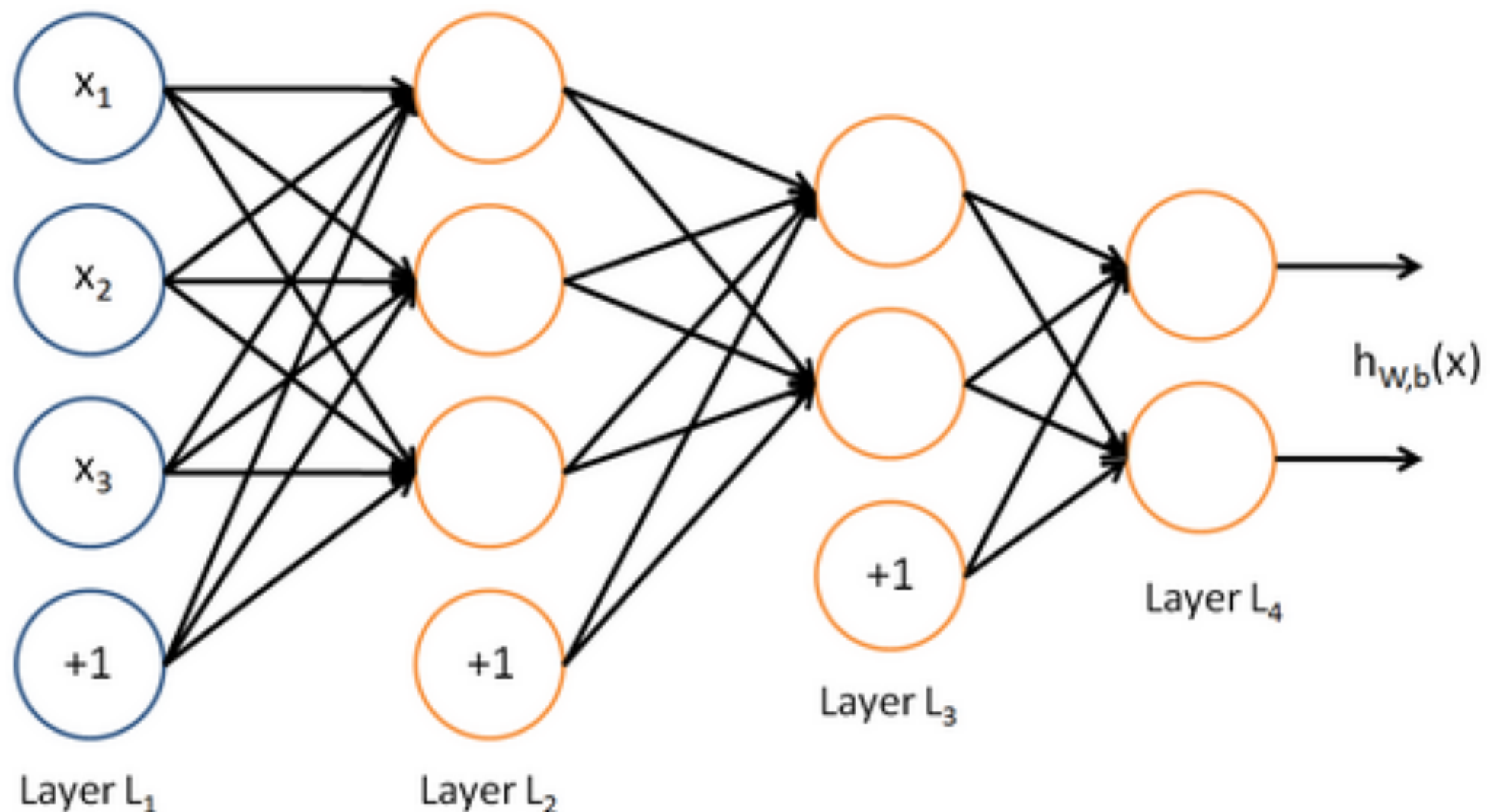


It is the loss function that will direct what the intermediate hidden variables should be, so as to do a good job at predicting the targets for the next layer, etc.

NN: like several intermediate logregs

Before we know it, we have a multilayer neural network....

a.k.a. **feedforward network** (see INLP on terminology)



Nonlinear activations

- “Squash functions”!

- Logistic / Sigmoid

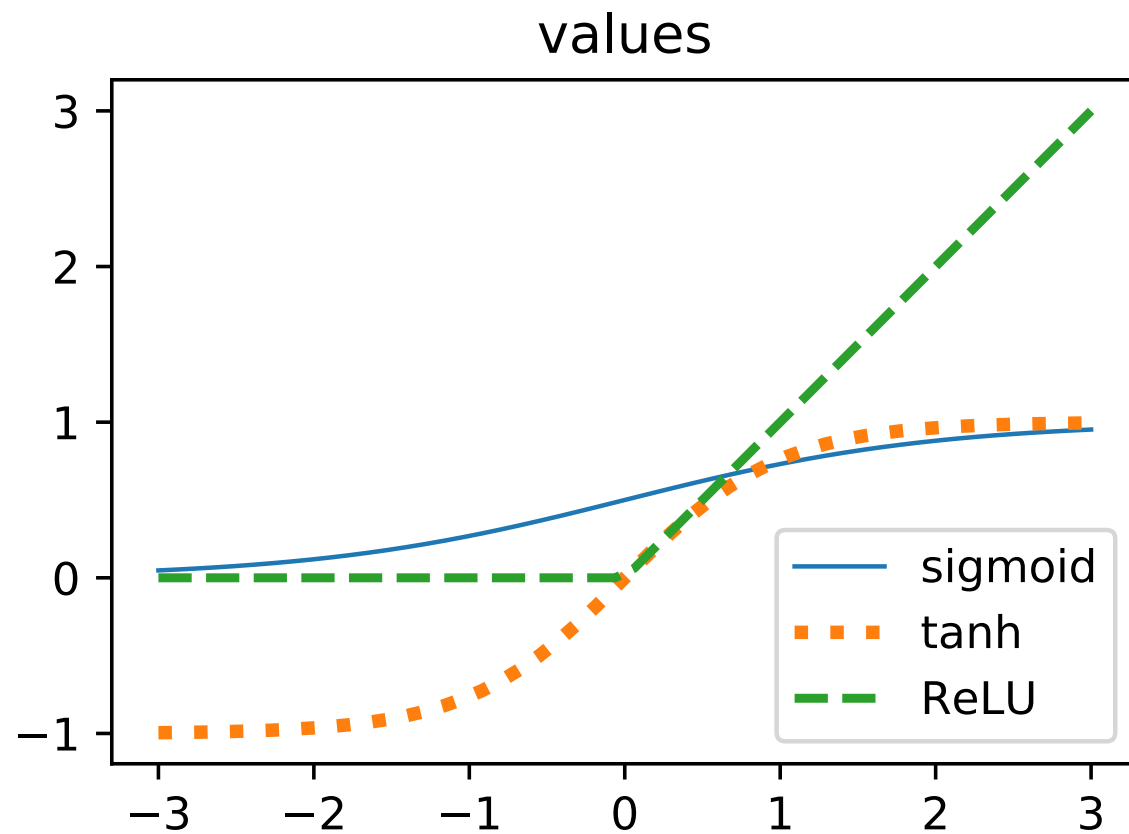
$$f(x) = \frac{1}{1 + e^{-x}} \quad (1)$$

- tanh

$$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1 \quad (2)$$

- ReLU

$$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases} \quad (3)$$



is a multi-layer neural network with no nonlinearities
(i.e., f is the identity $f(\mathbf{x}) = \mathbf{x}$)
more powerful than a one-layer network?

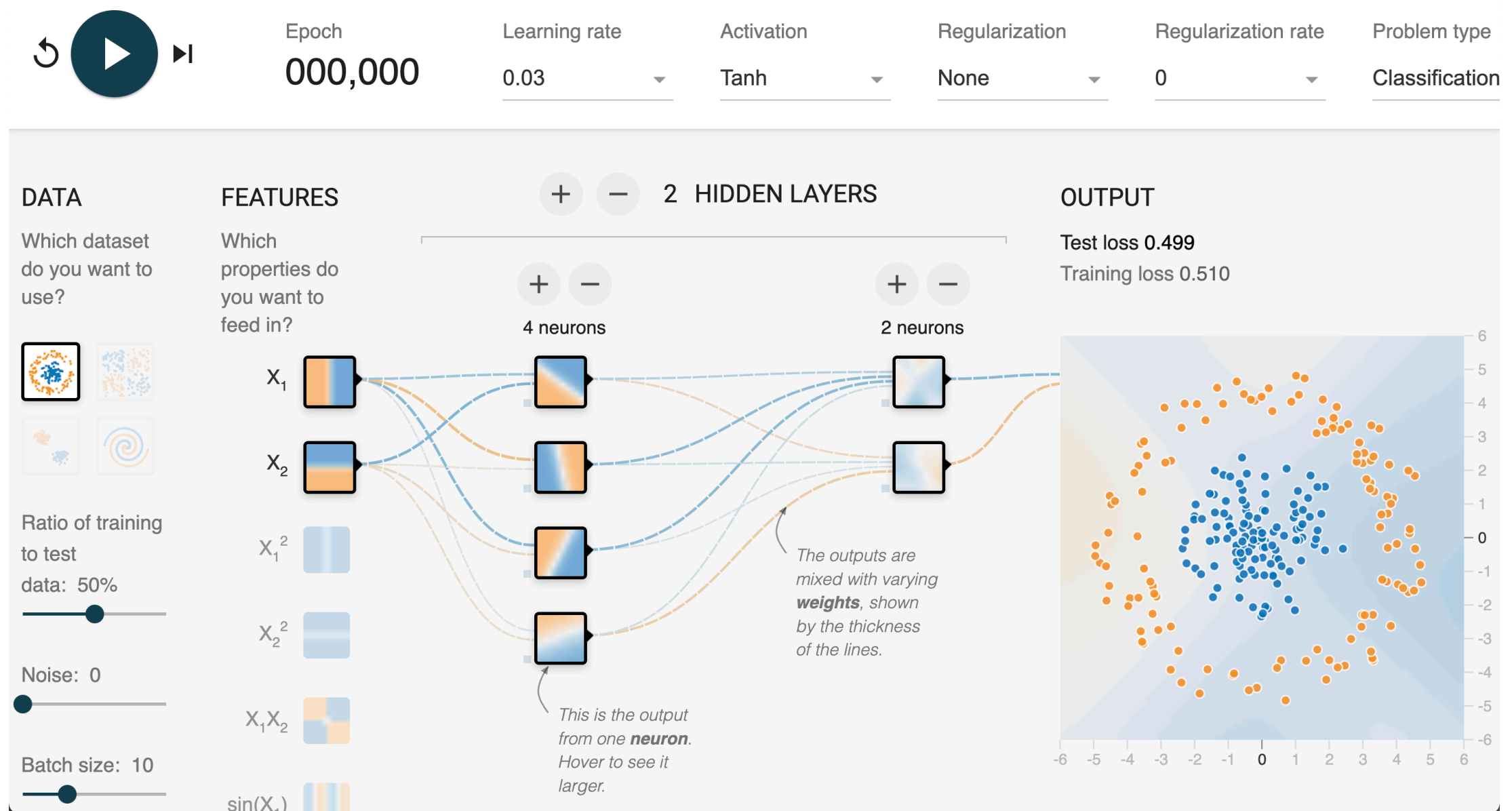
is a multi-layer neural network with no nonlinearities
(i.e., f is the identity $f(\mathbf{x}) = \mathbf{x}$)
more powerful than a one-layer network?

No! You can just compile all of the layers into a single transformation!

$$y = f(W_3 f(W_2 f(W_1 x))) = Wx$$

Demo

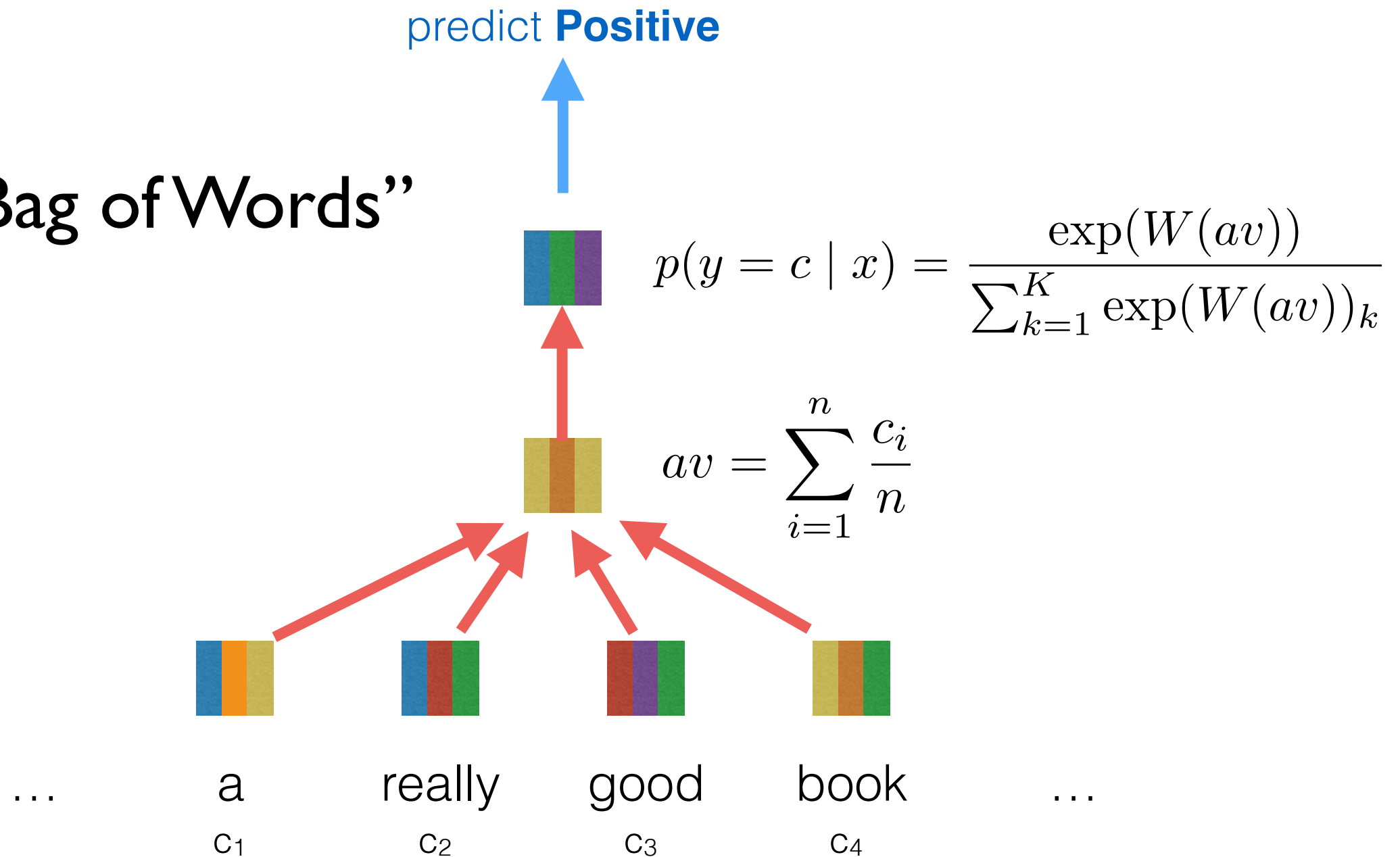
- <https://playground.tensorflow.org/>



- It's easy to create different neural network architectures, and execute gradient descent learning for arbitrary networks, via *backpropagation*
- e.g. the PyTorch library for Python
- Illustration: deep averaging models for text classification

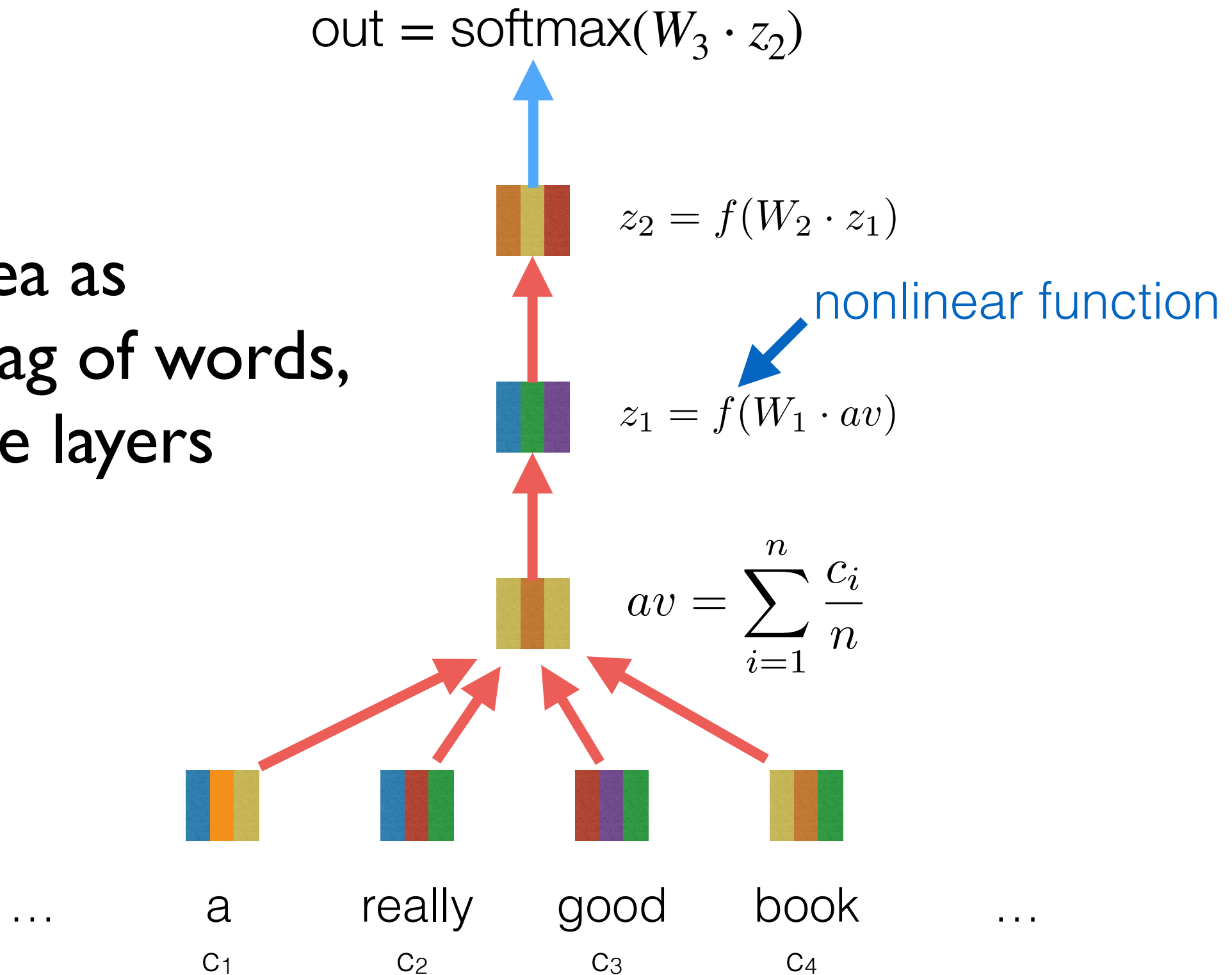
“bag of embeddings”

“Neural Bag of Words”



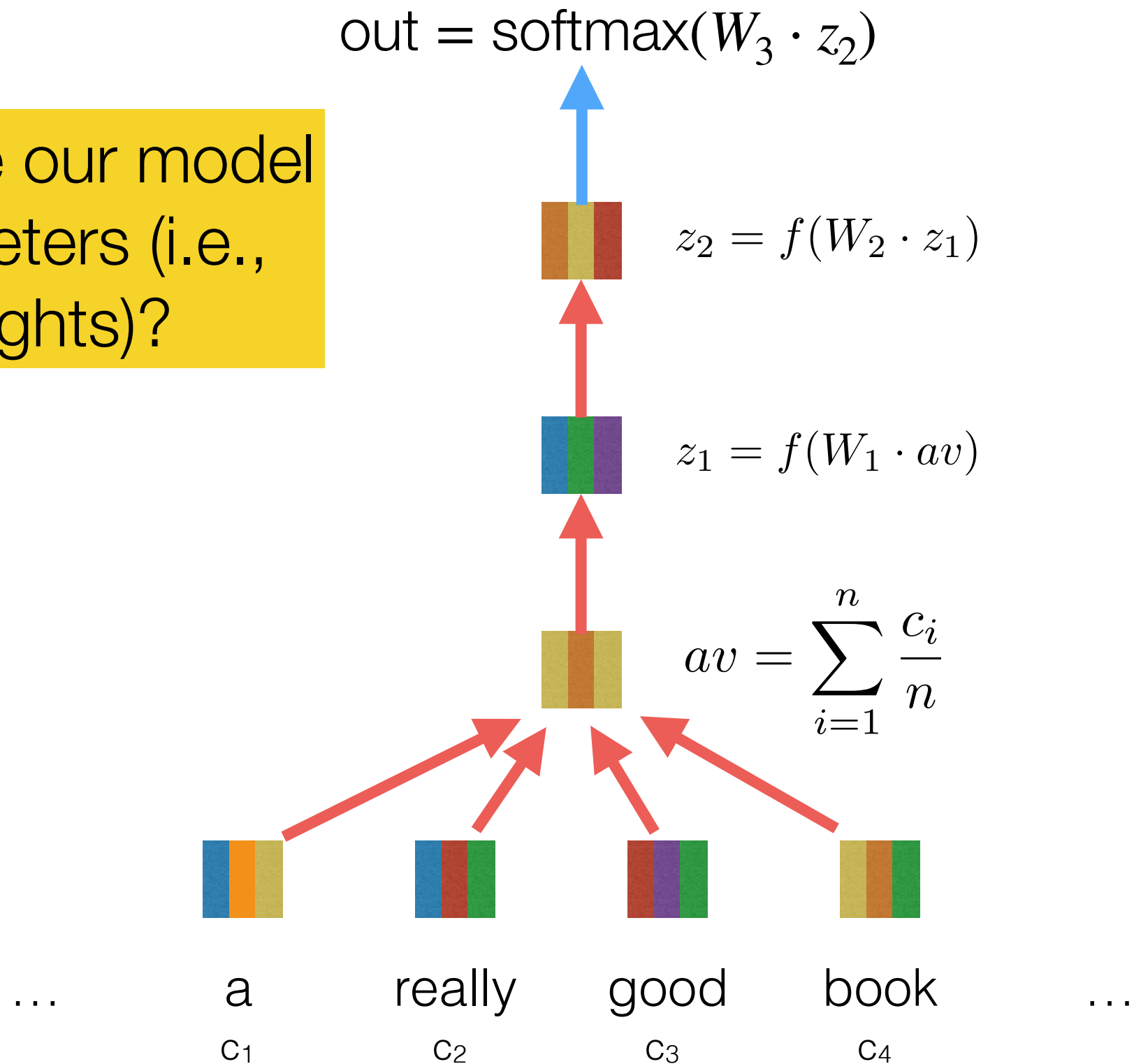
deep averaging networks

Same idea as
neural bag of words,
but more layers

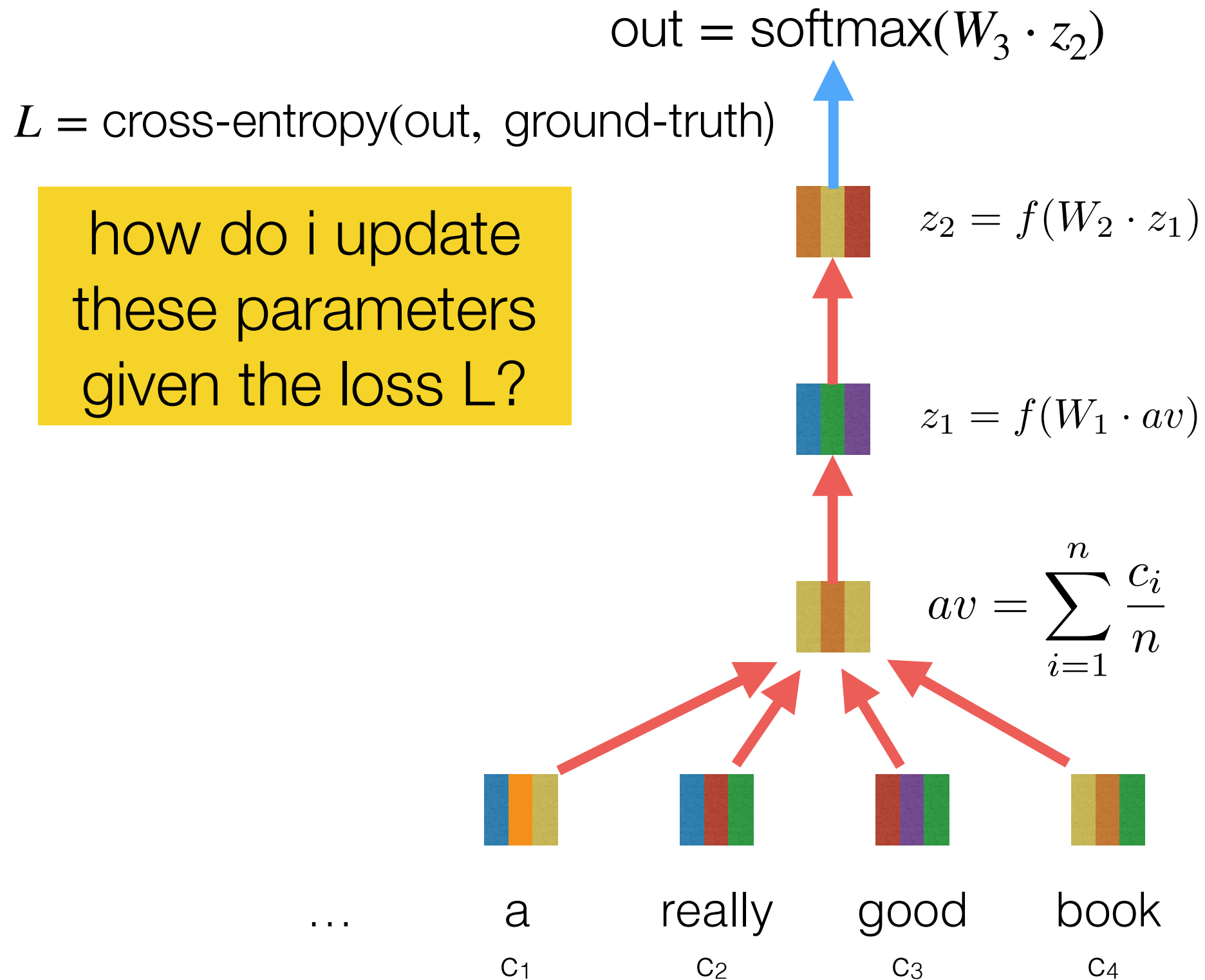


deep averaging networks

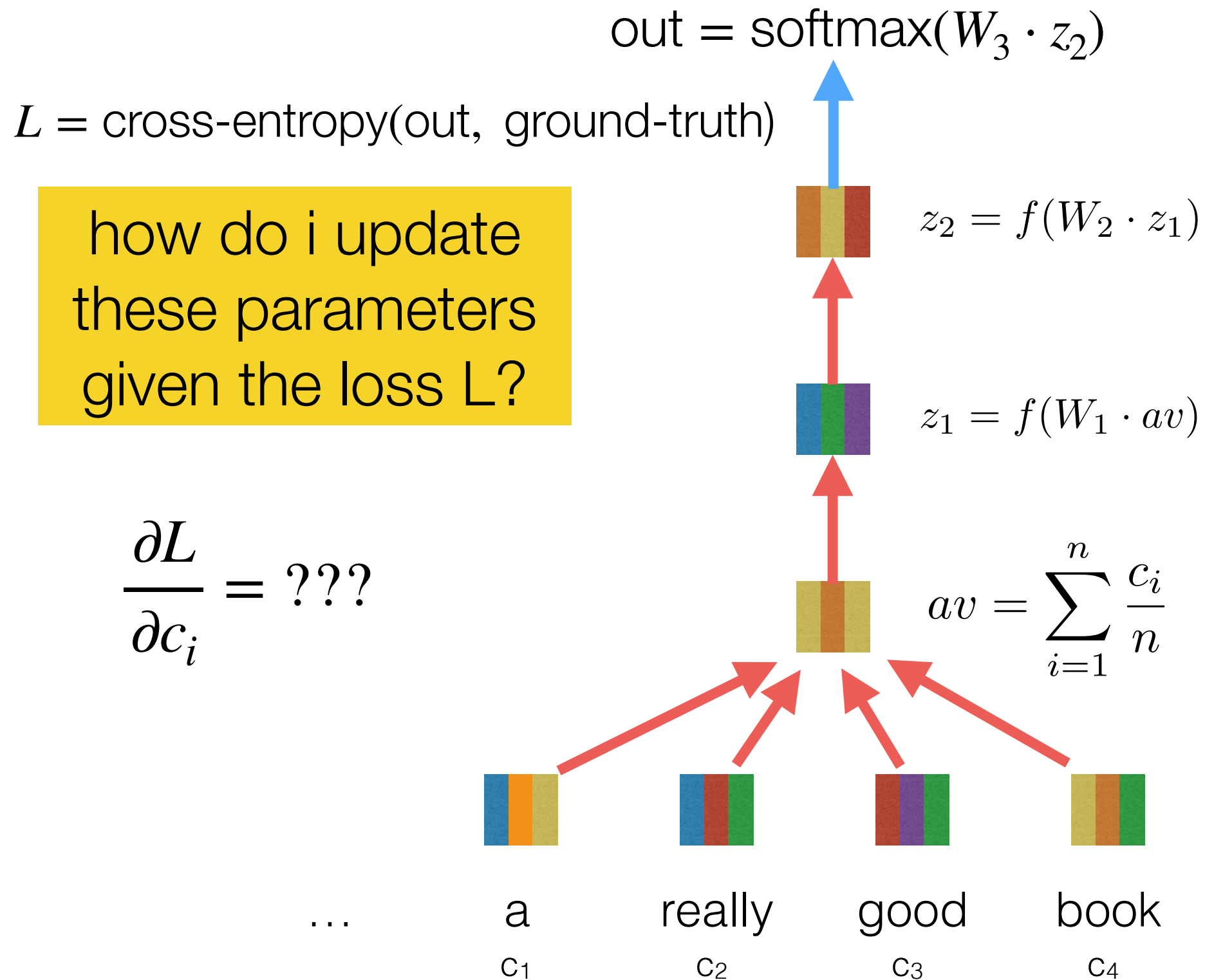
what are our model parameters (i.e., weights)?



deep averaging networks



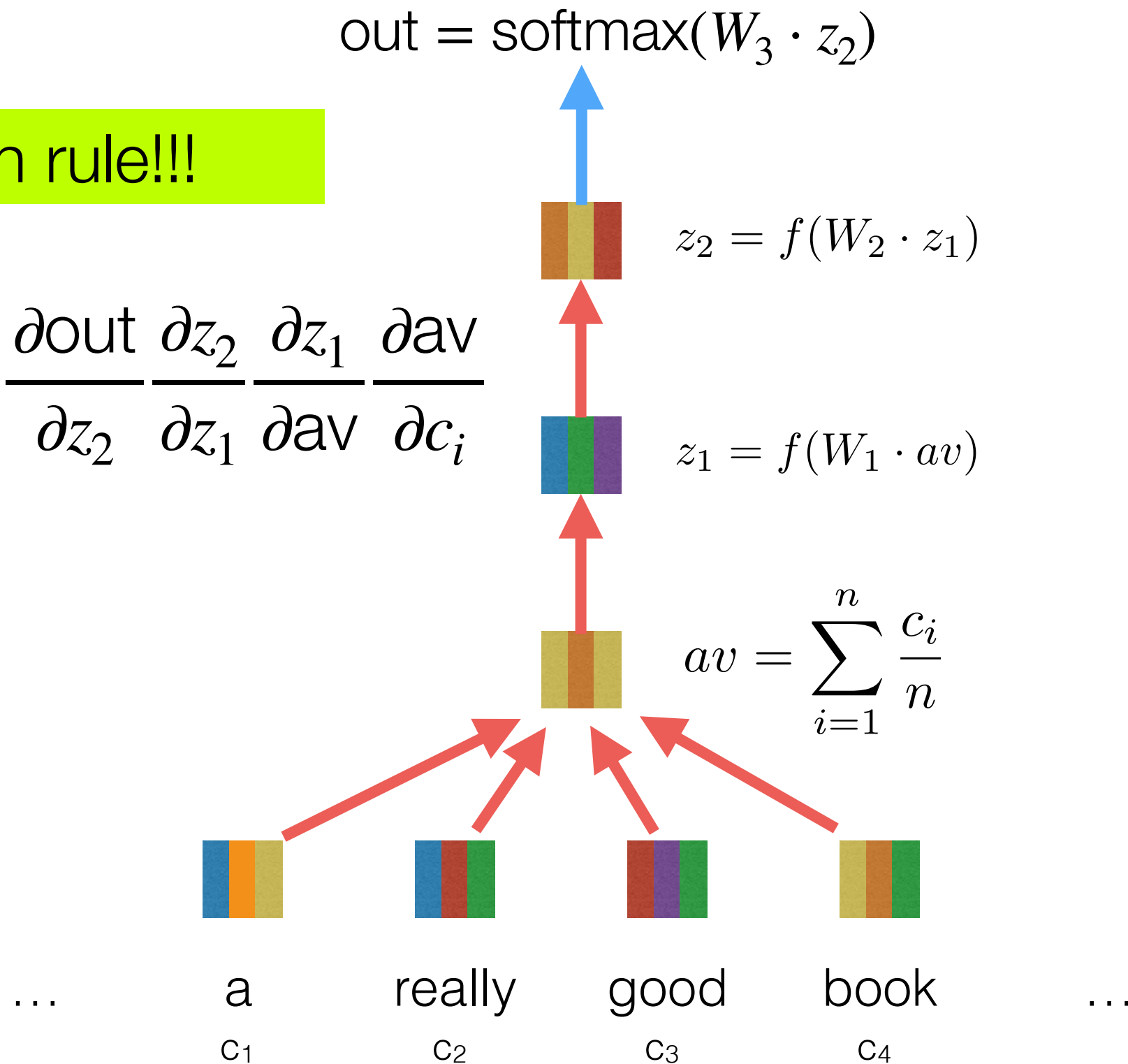
deep averaging networks



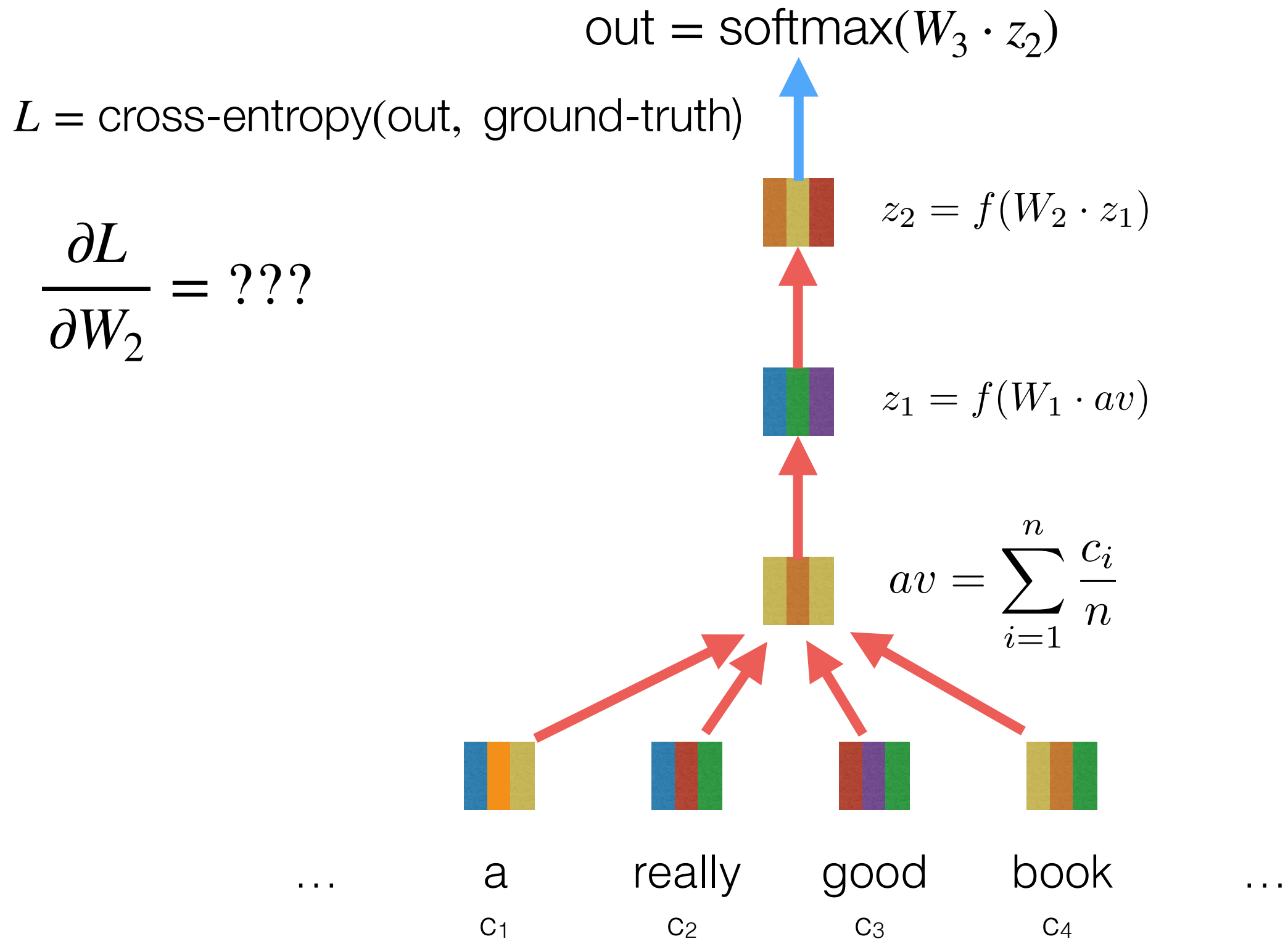
deep averaging networks

chain rule!!!

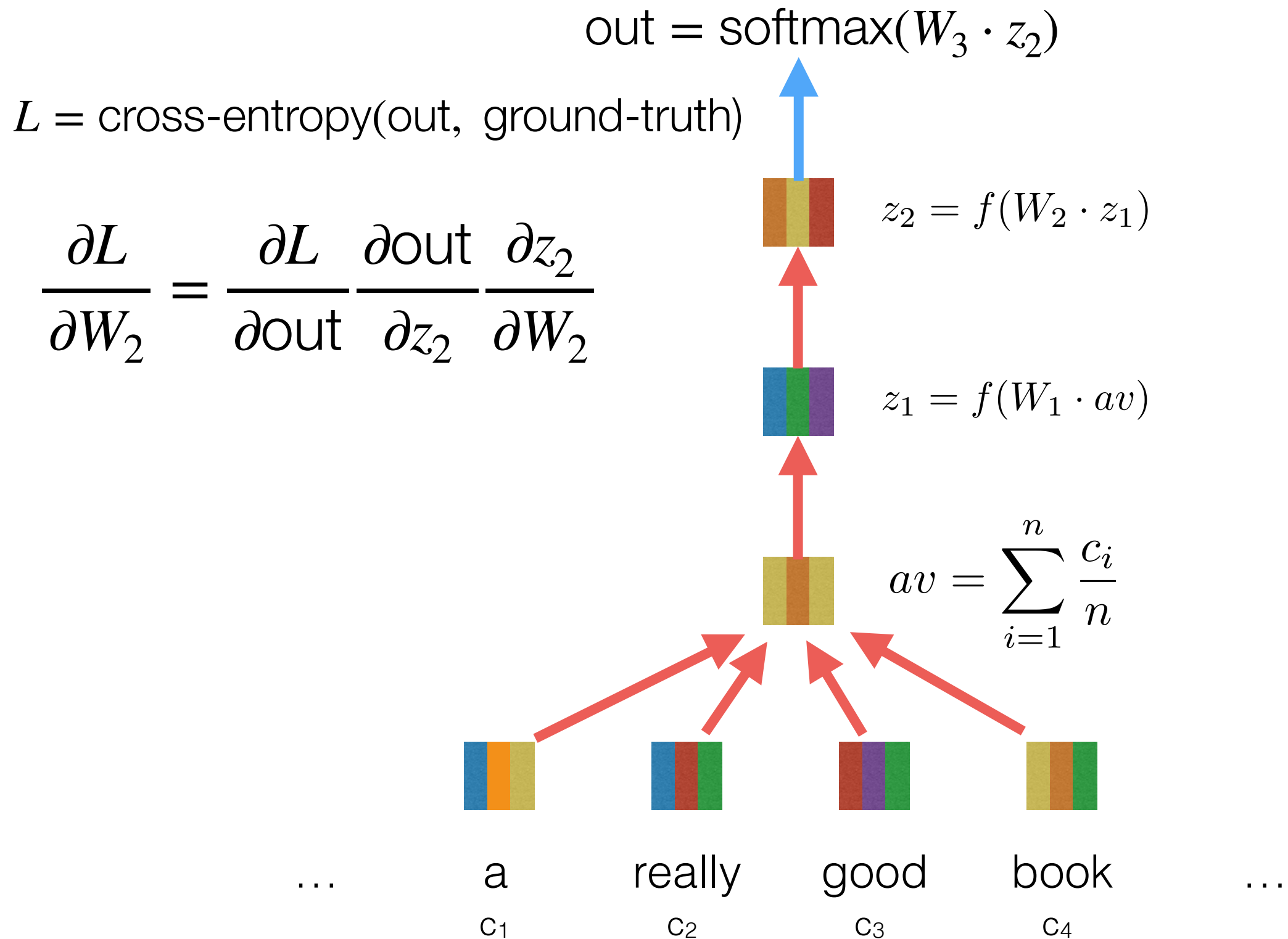
$$\frac{\partial L}{\partial c_i} = \frac{\partial L}{\partial \text{out}} \frac{\partial \text{out}}{\partial z_2} \frac{\partial z_2}{\partial z_1} \frac{\partial z_1}{\partial av} \frac{\partial av}{\partial c_i}$$



deep averaging networks



deep averaging networks



backpropagation

- use the chain rule to compute partial derivatives w/ respect to each parameter
- trick: re-use derivatives computed for higher layers to compute derivatives for lower layers!

$$\frac{\partial L}{\partial c_i} = \frac{\partial L}{\partial \text{out}} \frac{\partial \text{out}}{\partial z_2} \frac{\partial z_2}{\partial z_1} \frac{\partial z_1}{\partial \text{av}} \frac{\partial \text{av}}{\partial c_i}$$

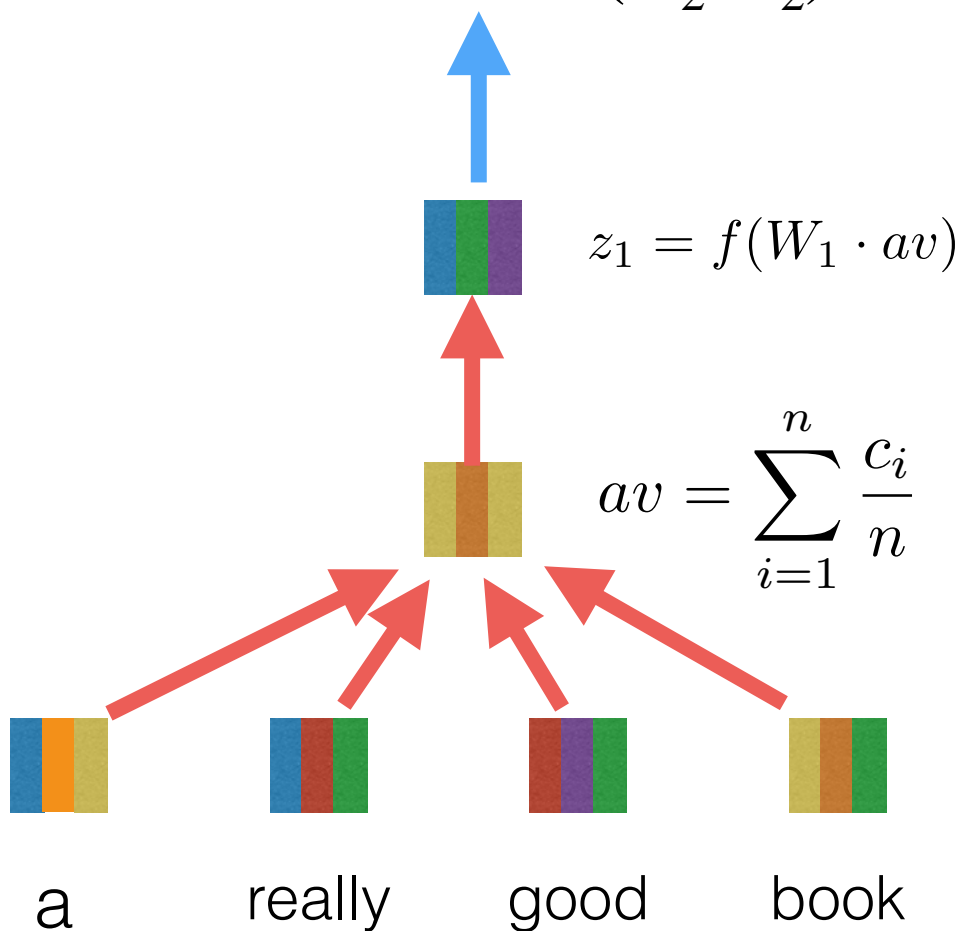
$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial \text{out}} \frac{\partial \text{out}}{\partial z_2} \frac{\partial z_2}{\partial W_2}$$

deep learning frameworks make building NNs super easy!

$$\text{out} = \text{softmax}(W_2 \cdot z_2)$$

$$z_1 = f(W_1 \cdot av)$$

$$av = \sum_{i=1}^n \frac{c_i}{n}$$



set up the network

```
def __init__(self, n_classes, vocab_size, emb_dim=300,
              n_hidden_units=300):
    super(DanModel, self).__init__()
    self.n_classes = n_classes
    self.vocab_size = vocab_size
    self.emb_dim = emb_dim
    self.n_hidden_units = n_hidden_units
    self.embeddings = nn.Embedding(self.vocab_size,
                                    self.emb_dim)
    self.classifier = nn.Sequential(
        nn.Linear(self.n_hidden_units,
                  self.n_hidden_units),
        nn.ReLU(),
        nn.Linear(self.n_hidden_units,
                  self.n_classes))
    self._softmax = nn.Softmax()
```

deep learning frameworks make building NNs super easy!

$$\text{out} = \text{softmax}(W_2 \cdot z_2)$$

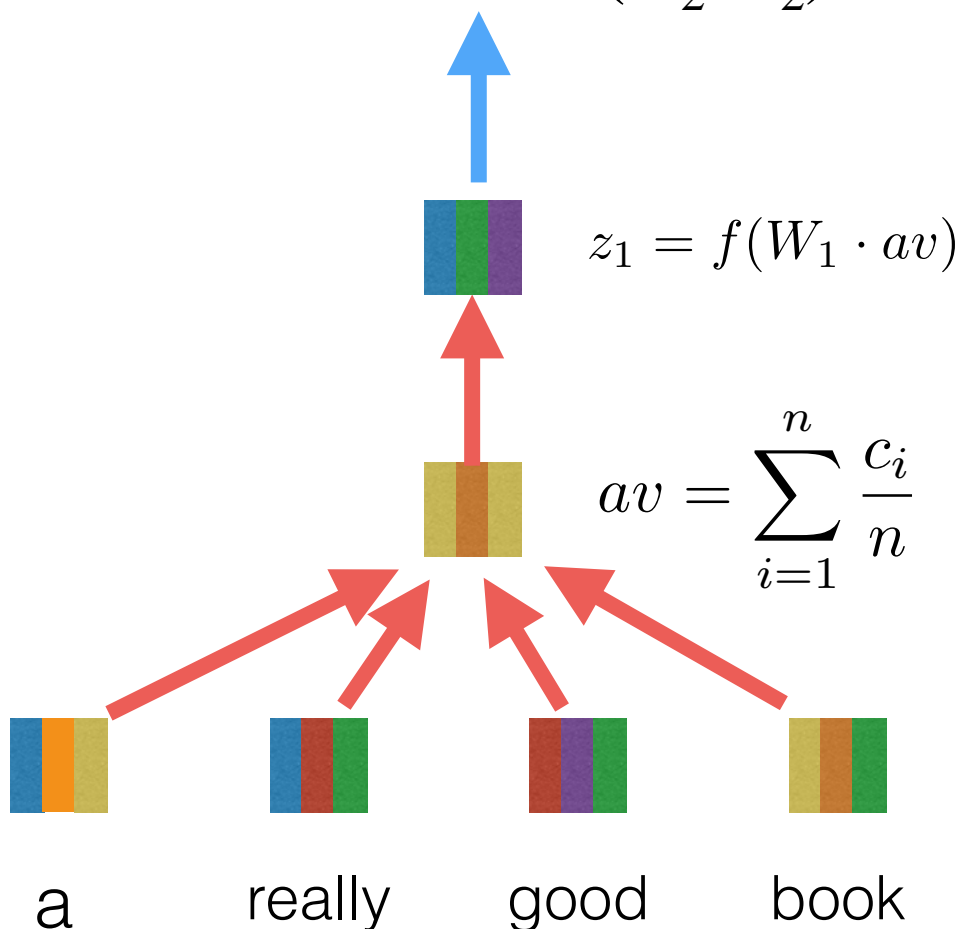
do a forward pass to compute prediction

```
def forward(self, batch, probs=False):
    text = batch['text']['tokens']
    length = batch['length']
    text_embed = self._word_embeddings(text)
    # Take the mean embedding. Since padding results
    # in zeros its safe to sum and divide by length
    encoded = text_embed.sum(1)
    encoded /= lengths.view(text_embed.size(0), -1)

    # Compute the network score predictions
    logits = self.classifier(encoded)
    if probs:
        return self._softmax(logits)
    else:
        return logits
```

$$z_1 = f(W_1 \cdot av)$$

$$av = \sum_{i=1}^n \frac{c_i}{n}$$



deep learning frameworks make building NNs super easy!

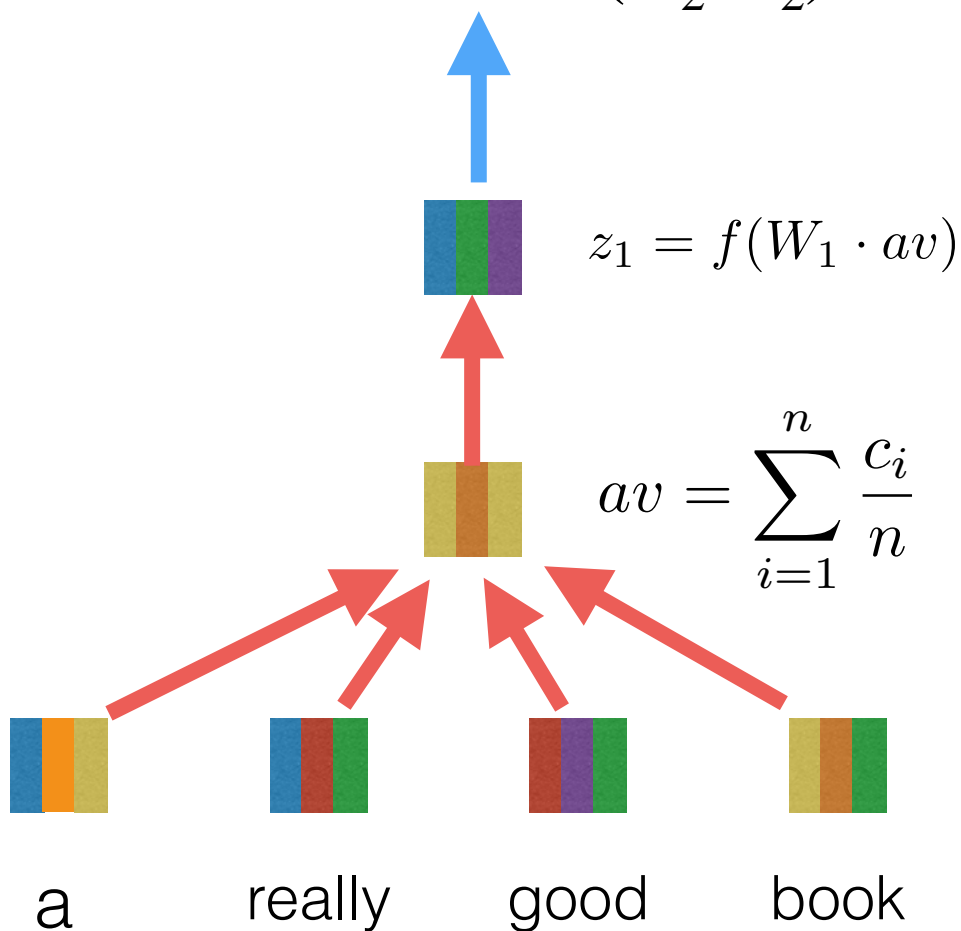
$$\text{out} = \text{softmax}(W_2 \cdot z_2)$$

do a backward pass to update weights

```
def _run_epoch(self, batch_iter, train=True):  
    self._model.train()  
    for batch in batch_iter:  
        model.zero_grad()  
        out = model(batches)  
        batch_loss = criterion(out,  
                                batch['label'])  
        batch_loss.backward()  
        self.optimizer.step()
```

$$z_1 = f(W_1 \cdot av)$$

$$av = \sum_{i=1}^n \frac{c_i}{n}$$



deep learning frameworks make building NNs super easy!

out = softmax($W_2 \cdot z_2$)

$$z_1 = f(W_1 \cdot av)$$

$$av = \sum_{i=1}^n \frac{c_i}{n}$$

do a backward pass to update weights

```
def _run_epoch(self, batch_iter, train=True):  
    self._model.train()  
    for batch in batch_iter:  
        model.zero_grad()  
        out = model(batches)  
        batch_loss = criterion(out,  
                                batch['label'])  
        batch_loss.backward()  
        self.optimizer.step()
```

that's it! no need to compute gradients by hand!

How NN models are used

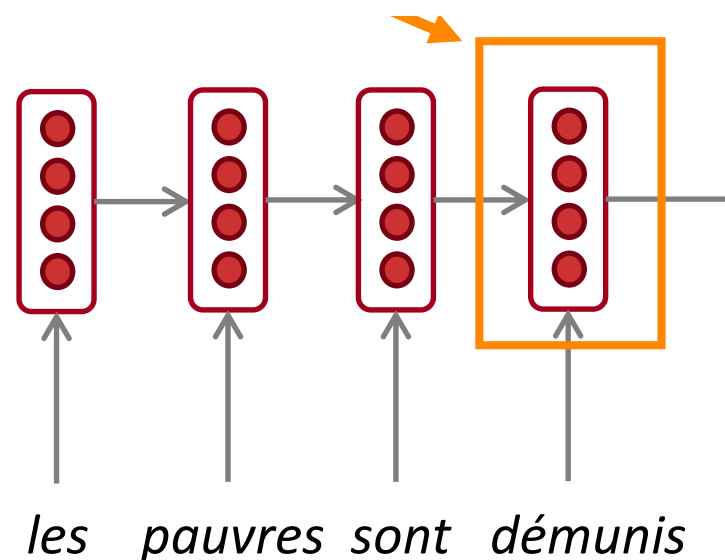
- For classification or tagging:
 - Direct supervised training
 - Pretrain then fine-tune pipeline
- Or: directly use an NN LM for generation

NN architectures

- We need neural network models that can process token-by-token
- Major components
 - Recurrent neural networks (RNNs)
 - Attention mechanism (softmax over tokens)
 - Self-attention ("Transformers"; next lecture)
 - This is basically the entire model behind BERT, GPT, Claude, Gemini, ... all standard NN NLP models today!

Recurrent neural networks

- Idea: beyond the original word embedding, every token has its own hidden state vector h_t , influenced by the previous state!



- Many applications
 - Next word prediction
 - Text classification
 - Translation...

Character-level RNN: Learning over time

- <https://karpathy.github.io/2015/05/21/rnn-effectiveness/>
RNN trained one character at a time
- Training on Tolstoy, War and Peace
- At iteration 100:
tyntd-iafhatawiaoirdemot lytdws e ,tfti, astai f ogoh
eoase rrranbyne 'nhthnee e plia tklrge t o idoe ns,smtt
h ne etie h,hregtrs nigtike,aoaenns lng
- At 300 iterations:
"Tmont thithey" fomesscerliund Keushey. Thom here
sheulke, anmerenith ol sivh l lalterthend Bleipile shuwyy fil
on aseterlome coaniogennc Phe lism thond hon at.
MeiDimorotion in ther thize."

Character-level RNN: Learning over time

- <https://karpathy.github.io/2015/05/21/rnn-effectiveness/>
RNN trained one character at a time
- Training
100: Mostly gibberish, but starting to get an idea of word boundaries. sometimes two spaces instead of one
tyntd-iafhatawiaoighrdemot lytdws e ,tfti, astai f ogoh
eoase rrranbyne 'nhthnee e plia tklrqd t o idoe ns,smtt
h ne etie h,hregtrs nigtike,aoaenns lng
- At 300
300: model starts to get an idea about quotes and periods:
"Tmont thithey" tomesscerliund Keushey. I hom here
sheulke, anmerenith ol sivh I lalterthend Bleipile shuwyy fil
on aseterlome coaniogennc Phe lism thond hon at.
MeiDimorotion in ther thize."

Character-level RNN: Learning over time

- Iteration 500:
we counter. He stutn co des. His stanted out one ofler that concossions and was to gearang reay Jotrets and with fre colt of paitt thin wall. Which das stimn
- Iteration 700:
Aftair fall unsuch that the hall for Prince Velzonski's that me of her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort how, and Gogition is so overelical and ofter.
- About iteration 2000:
"Why do what that day," replied Natasha, and wishing to himself the fact the princess, Princess Mary was easier, fed in had oftened him. Pierre aking his soul came to the packs and drove up his father-in-law women.

Character-level RNN: Learning over time

- **500: some short, common words come out right**

we counter. He stutn co des. His stanted out one ofler that concossions and was to gearang reay Jotrets and with fre colt of paitt thin wall. Which das stimn

- **700: more and more English-like text**

Aftair fall unsuch that the hall for Prince Velzonski's that me of her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort how, and Gogition is so overelical and ofter.

- About iteration 2000:

"Why do what that day," replied Natasha, and wishing to himself the fact the princess, Princess Mary was easier, fed in had oftened him. Pierre aking his soul came to the packs and drove up his father-in-law women.

Structure awareness (one particular RNN hidden state)

Cell sensitive to position in line:

The sole importance of the crossing of the Berezina lies in the fact that it plainly and indubitably proved the fallacy of all the plans for cutting off the enemy's retreat and the soundness of the only possible line of action--the one Kutuzov and the general mass of the army demanded--namely, simply to follow the enemy up. The French crowd fled at a continually increasing speed and all its energy was directed to reaching its goal. It fled like a wounded animal and it was impossible to block its path. This was shown not so much by the arrangements it made for crossing as by what took place at the bridges. When the bridges broke down, unarmed soldiers, people from Moscow and women with children who were with the French transport, all--carried on by vis inertiae--pressed forward into boats and into the ice-covered water and did not, surrender.

Cell that turns on inside quotes:

"You mean to imply that I have nothing to eat out of.... On the contrary, I can supply you with everything even if you want to give dinner parties," warmly replied Chichagov, who tried by every word he spoke to prove his own rectitude and therefore imagined Kutuzov to be animated by the same desire.

Kutuzov, shrugging his shoulders, replied with his subtle penetrating smile: "I meant merely to say what I said."

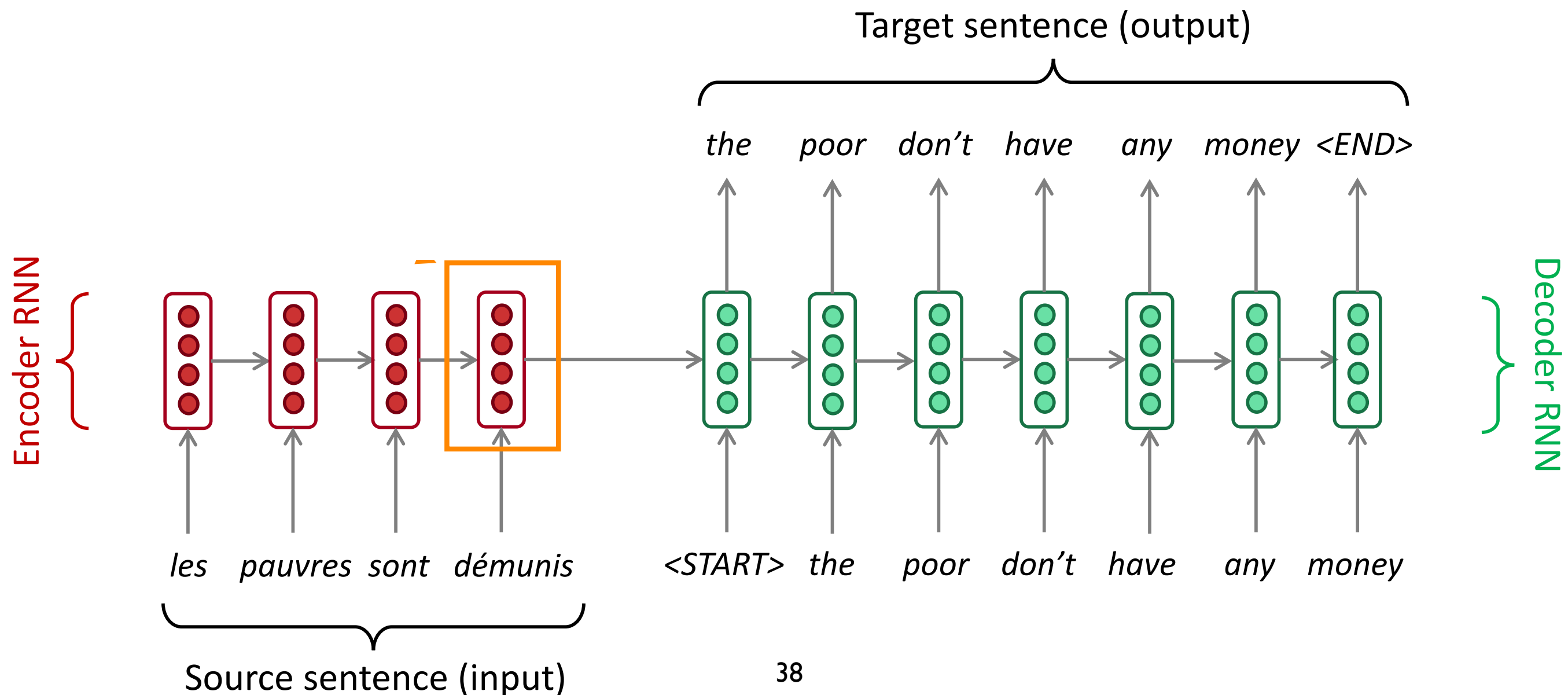
Cell that robustly activates inside if statements:

```
static int __dequeue_signal(struct sigpending *pending, sigset_t *mask,
                           siginfo_t *info)
{
    int sig = next_signal(pending, mask);
    if (sig) {
        if (current->notifier) {
            if (sigismember(current->notifier_mask, sig)) {
                if (!(current->notifier)(current->notifier_data)) {
                    clear_thread_flag(TIF_SIGPENDING);
                    return 0;
                }
            }
        }
        collect_signal(sig, pending, info);
    }
    return sig;
}
```

A large portion of cells are not easily interpretable. Here is a typical example:

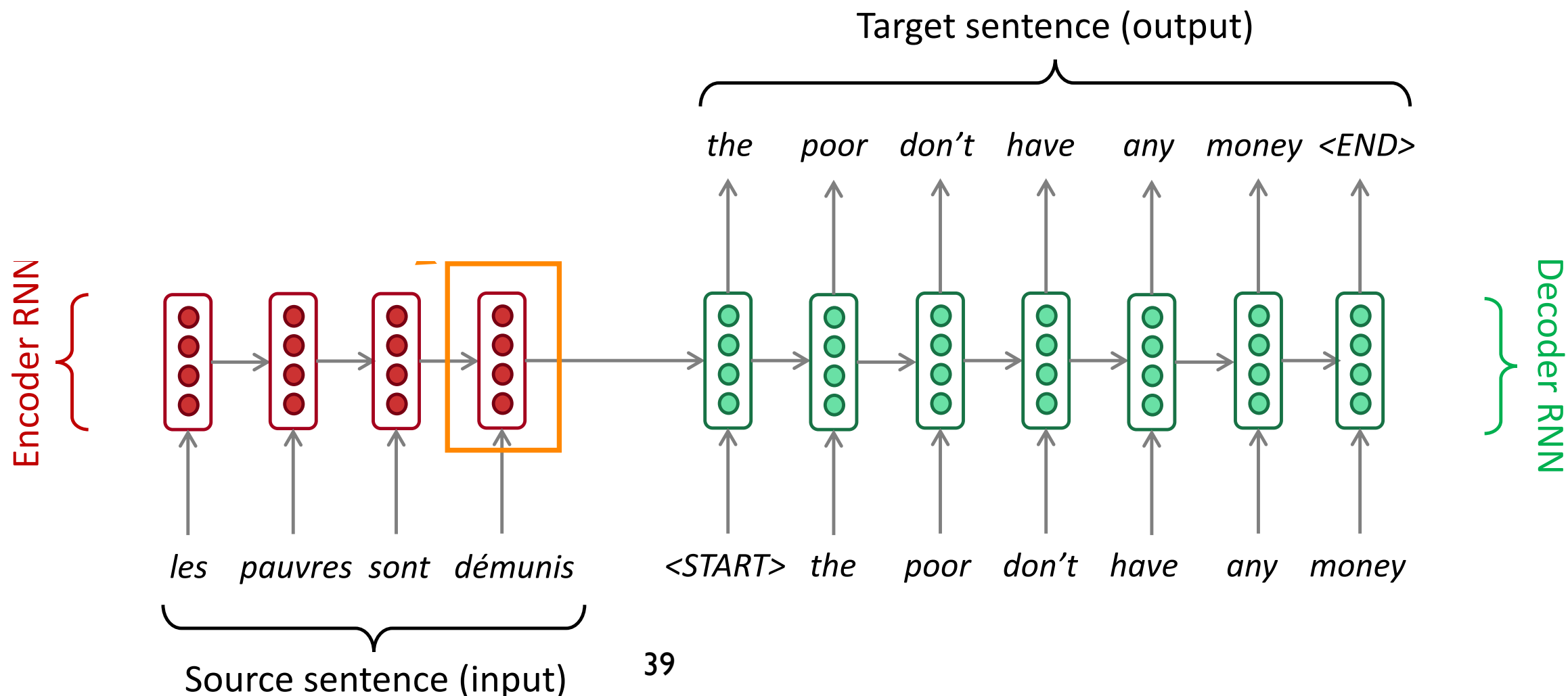
```
/* Unpack a filter field's string representation from user-space
 * buffer. */
char *audit_unpack_string(void **bufp, size_t *remain, size_t len)
{
    char *str;
    if (!*bufp || (len == 0) || (len > *remain))
        return ERR_PTR(-EINVAL);
    /* Of the currently implemented string fields, PATH_MAX
     * defines the longest valid length.
     */
}
```

Sequence to sequence models



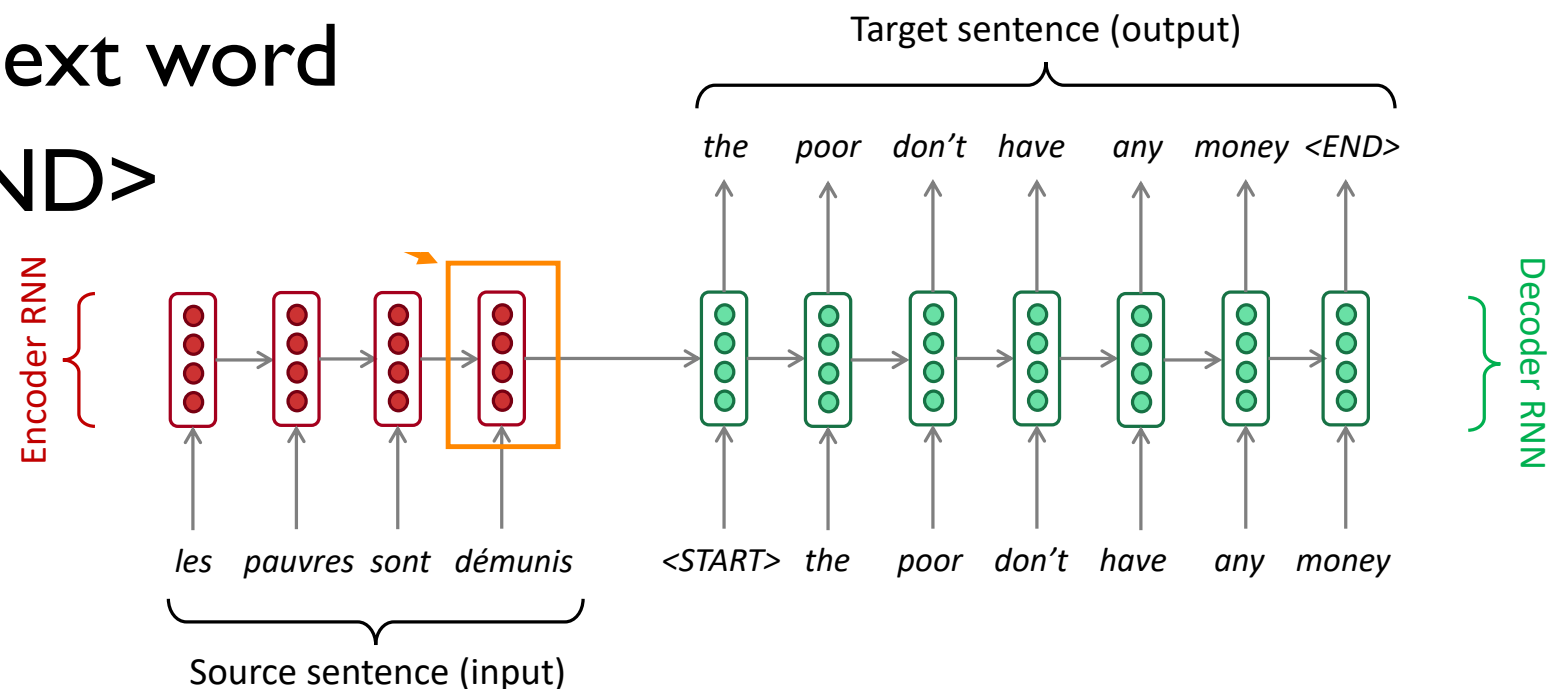
Sequence to sequence models

- Encoder-Decoder RNN for machine translation:
 - First half of the model: **Encoder**.
Reads in a sentence in Language 1.
 - Second half of the model: **Decoder**. Generates translation in language 2.



Sequence to sequence models

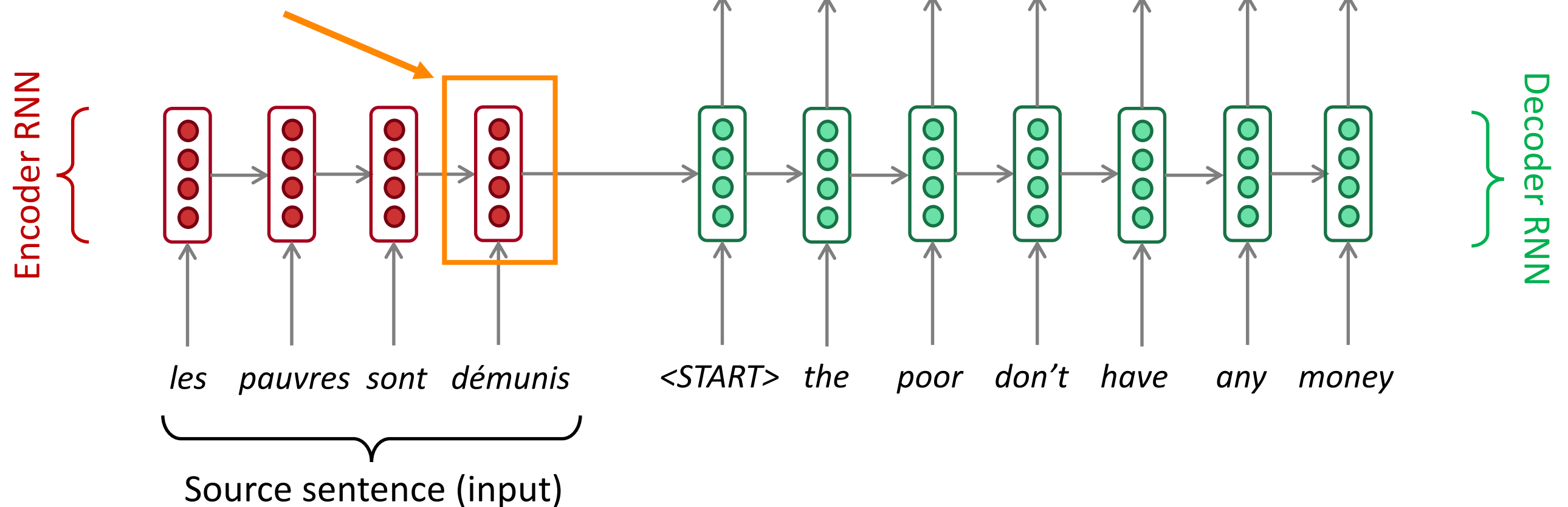
- **Encoder:** In each step, read in one word of input.
Aggregate content in hidden states
- Upon seeing the input <START>, switch to decode
- **Decoder:**
 - Read previous generated word
 - Generate next word
 - Stop at <END>



Sequence-to-sequence: the bottleneck problem

Encoding of the
source sentence.

This needs to capture *all*
information about the
source sentence.
Information bottleneck!



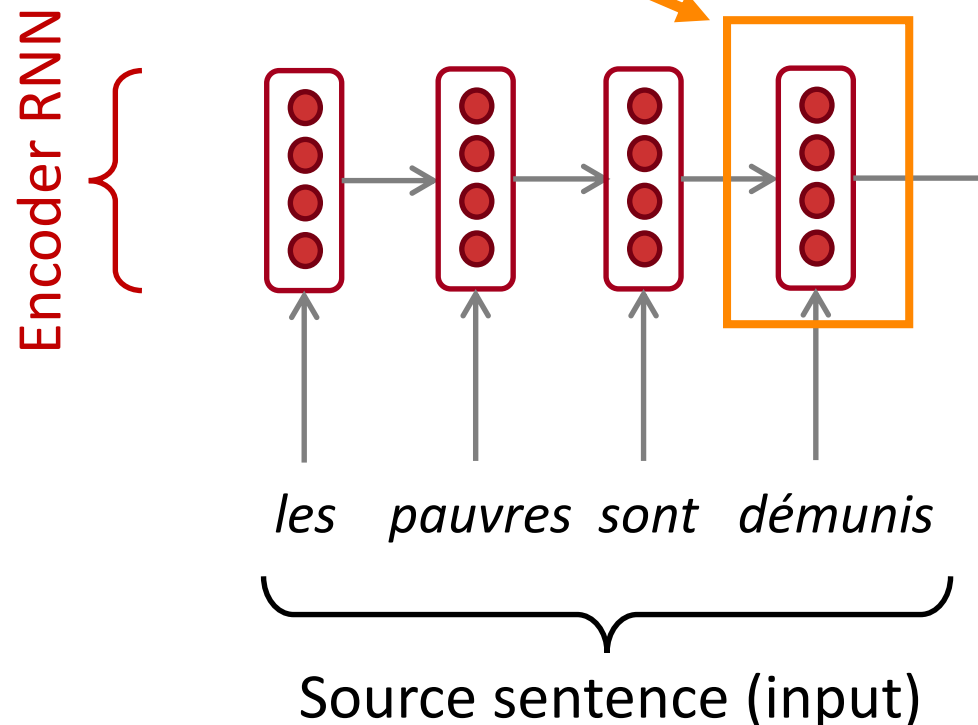
“you can’t cram the meaning
of a whole %&@#&ing
sentence into a single
\$*(&@ing vector!”

— Ray Mooney (famous NLP professor at UT Austin)

Idea: what if we use multiple vectors?

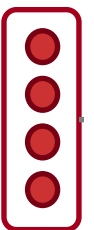
Encoding of the
source sentence.

This needs to capture *all*
information about the
source sentence.
Information bottleneck!



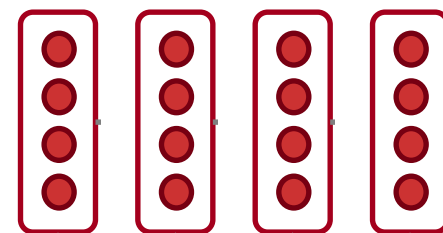
Instead of:

les pauvres sont démunis =



Let's try:

les pauvres sont démunis =



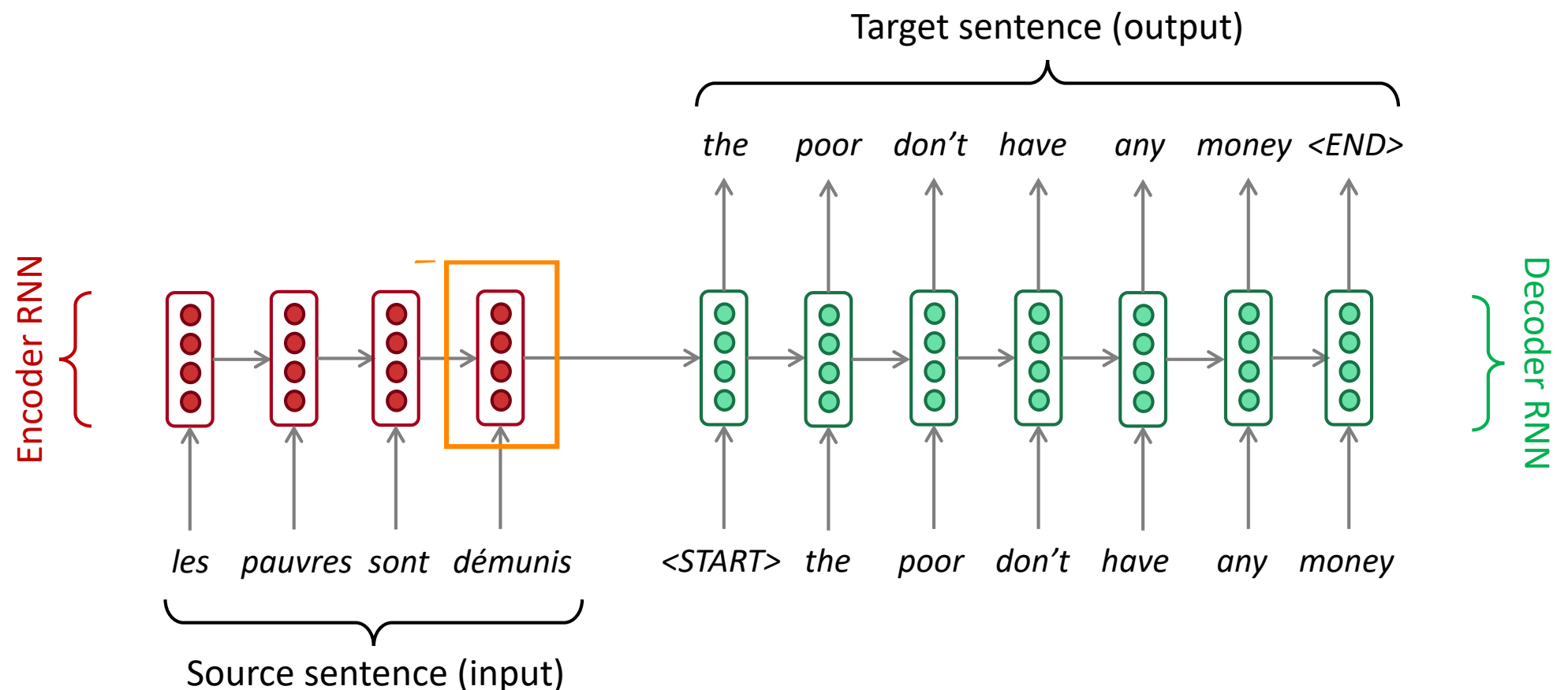
(all 4 hidden states!)

The solution: **attention**

- **Attention mechanisms** (Bahdanau et al., 2015) allow the decoder to focus on a particular part of the source sequence at each time step
 - Conceptually similar to *word alignments* in earlier MT models
 - For MT, can model differences in word order between languages

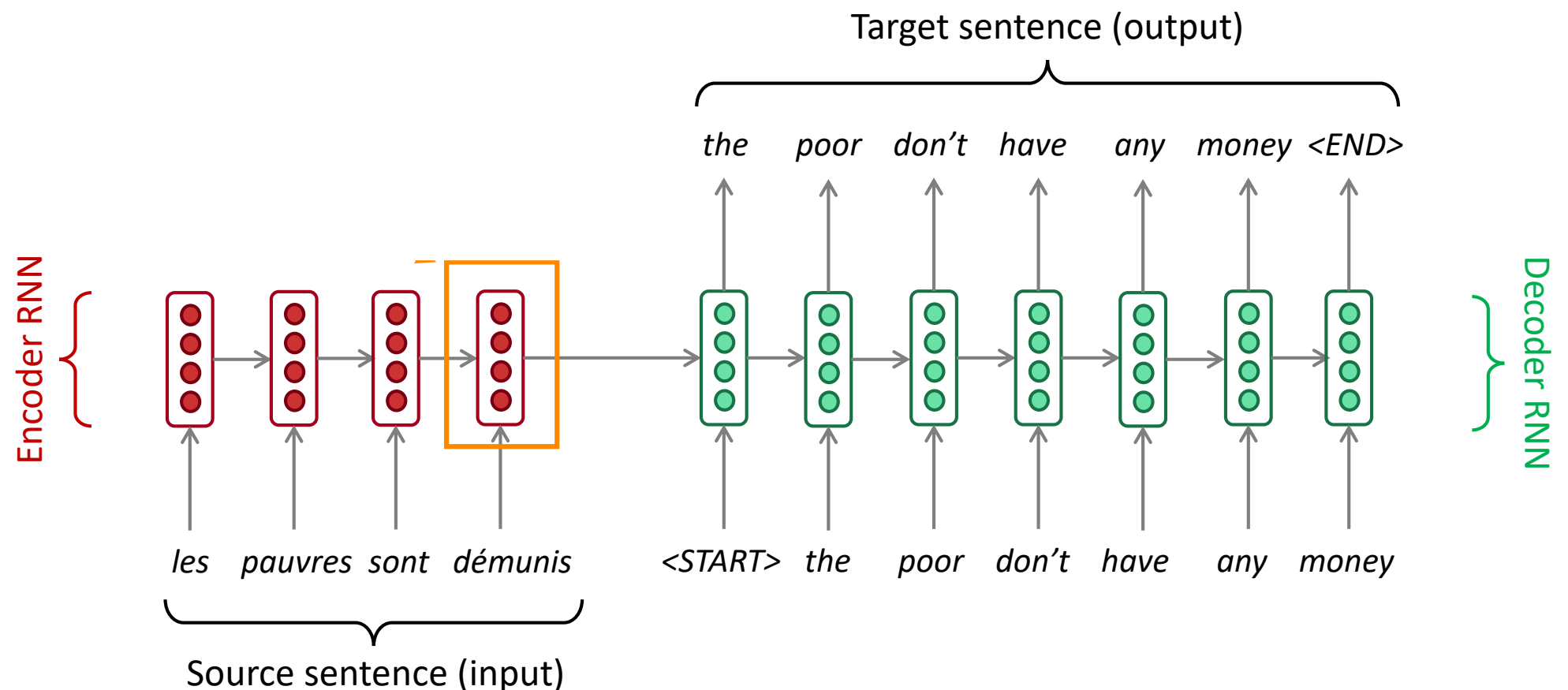
What do we want to do

- 2nd decoder state: just generated “the”, about to generate “poor”
- Use information from all encoder states
- Not all encoder states are equally important
- Most important: 2nd encoder state, after reading “pauvres”



What do we want to do

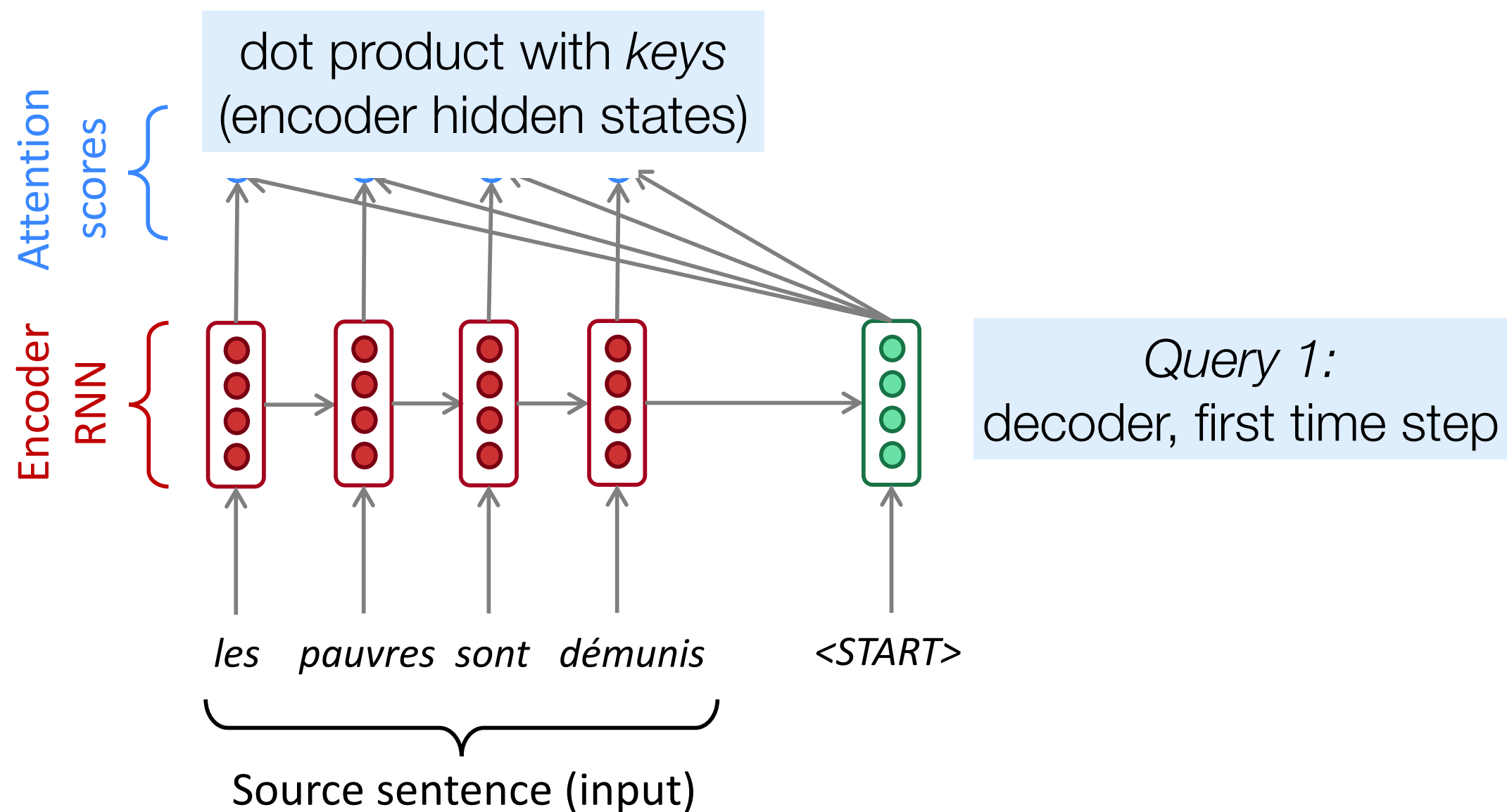
- Make a weighted sum of all encoder states
- Weighted by “attention”/ importance



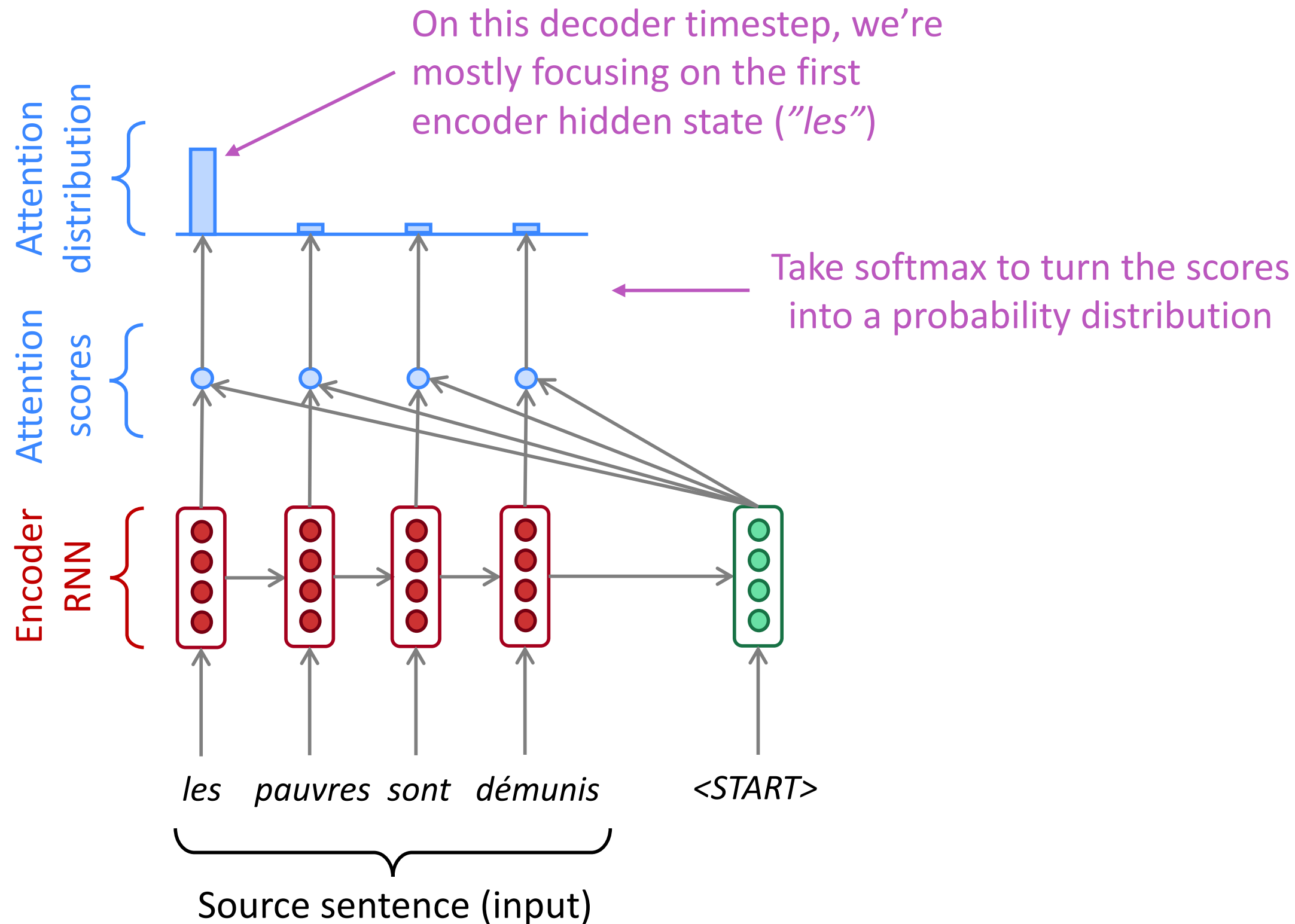
How does it work?

- MT encoder-decoder:
 - Query vector : current decoder hidden layer
 - Key vector: an encoder hidden layer
- Single *query* vector and multiple *key* vectors.
We want to score each query-key pair
 - Attention score based on query-key similarity
 - New representation = softmax-weighted average of encoder hidden layers

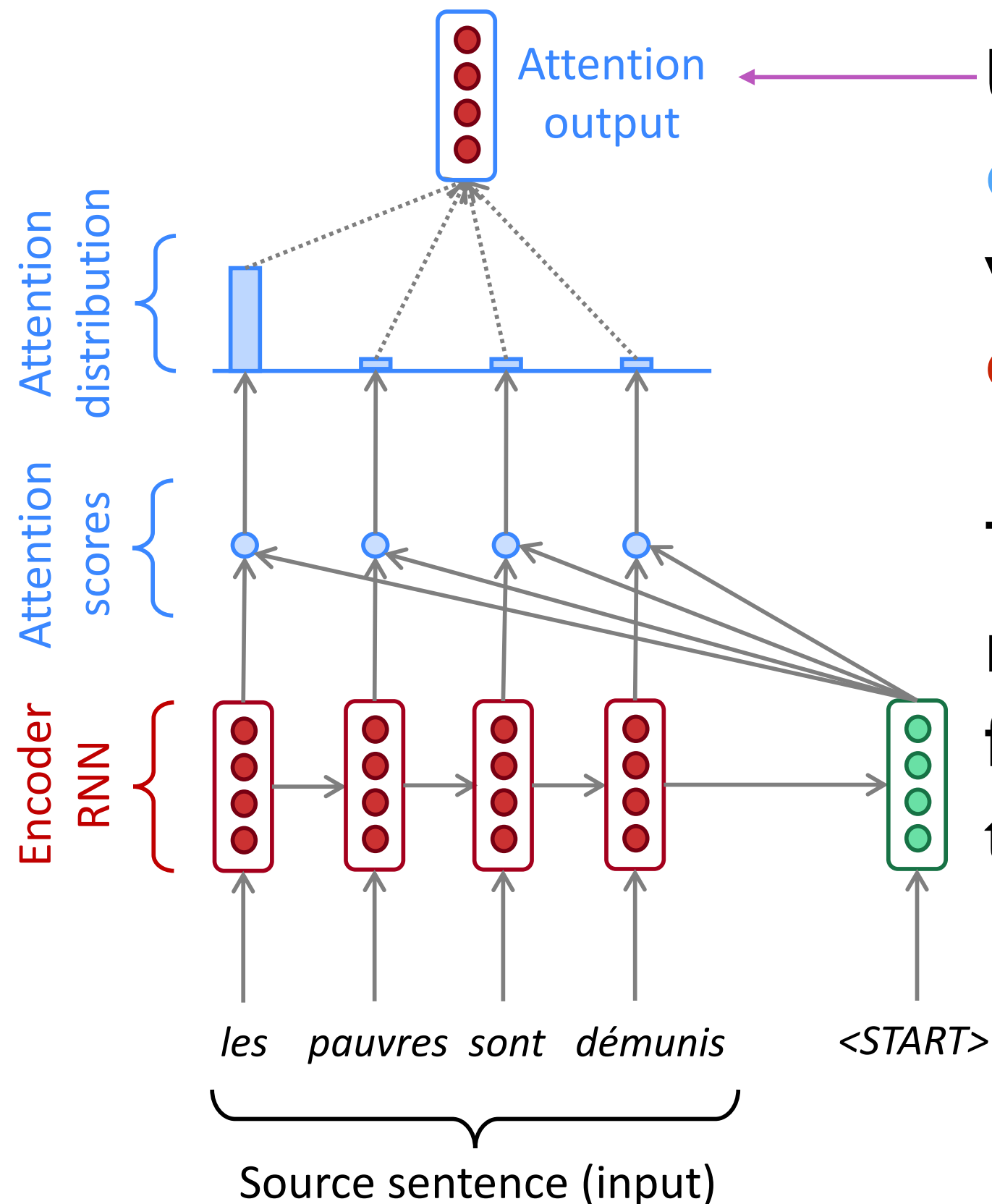
Sequence-to-sequence with attention



Sequence-to-sequence with attention



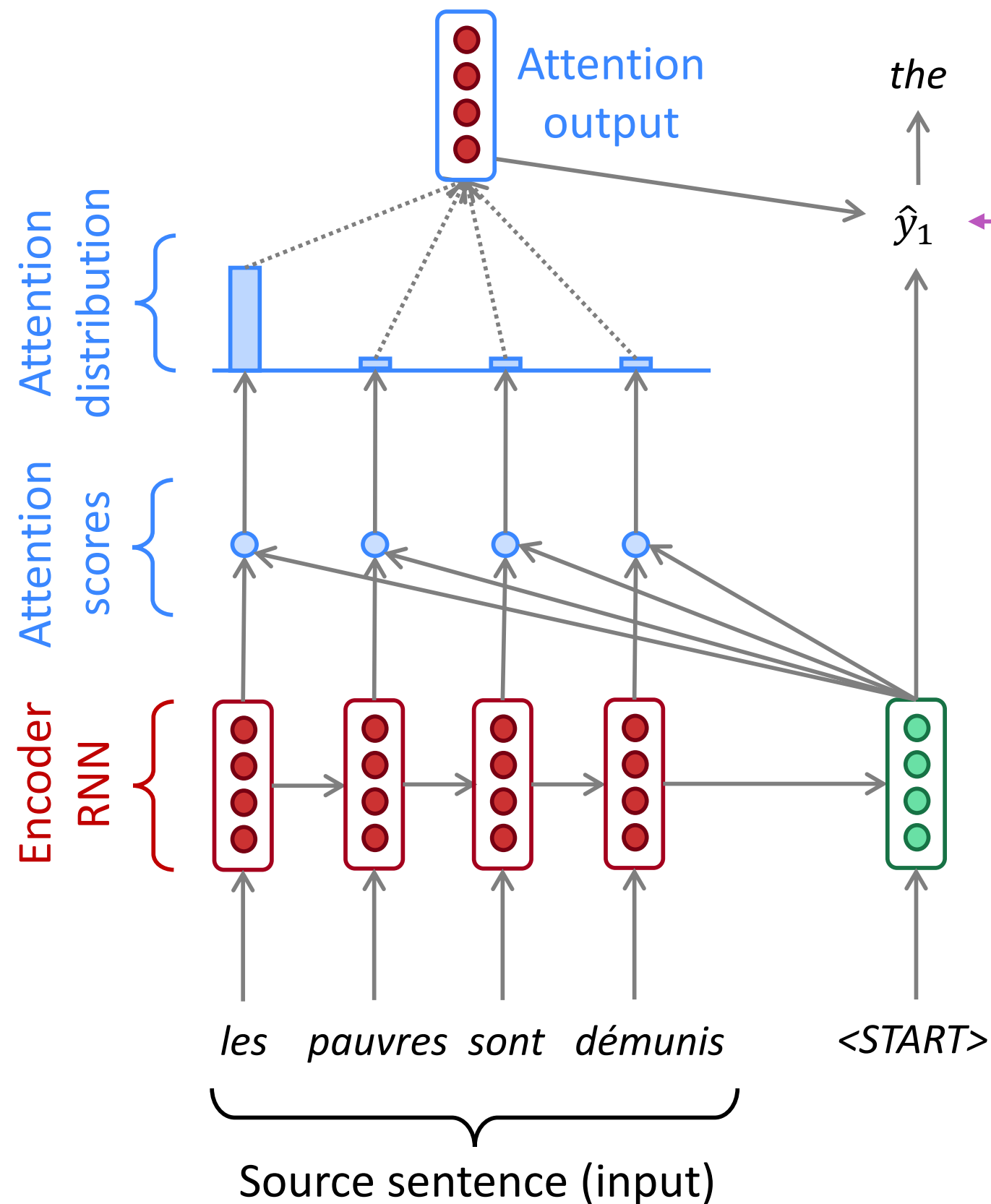
Sequence-to-sequence with attention



Use the **attention distribution** to take a weighted sum of the **encoder hidden states**.

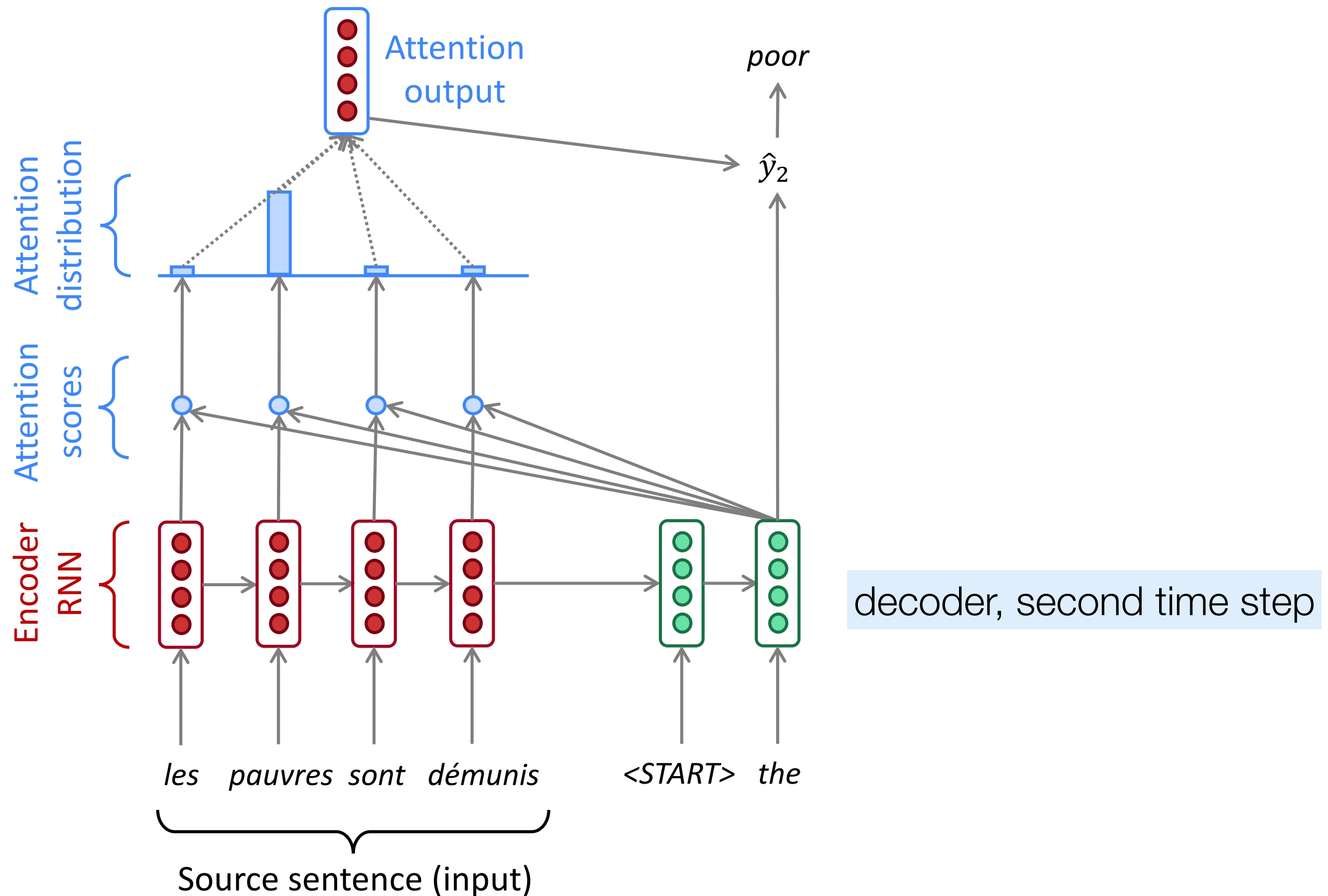
The attention output mostly contains information from the hidden states that received high attention.

Sequence-to-sequence with attention



Concatenate attention output with **decoder hidden state**, then use to compute $\hat{y}_1$ as before

Sequence-to-sequence with attention



Many variants of attention

- Dot product: $a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^T \mathbf{k}$ Luong et al., 2015
- Scaled dot product: $a(\mathbf{q}, \mathbf{k}) = \frac{\mathbf{q}^T \mathbf{k}}{\sqrt{|\mathbf{k}|}}$ Vaswani et al., 2017
- Attention score based on query-key similarity
- New representation = softmax-weighted average of token embeddings

Attention is great

- Attention significantly **improves NMT performance**
 - It's very useful to allow decoder to focus on certain parts of the source
- Attention **solves the bottleneck problem**
 - Attention allows decoder to look directly at source; bypass bottleneck
- Attention **helps with vanishing gradient problem**
 - Provides shortcut to faraway states
- Attention provides **some interpretability**
 - By inspecting attention distribution, we can see what the decoder was focusing on
 - We get **alignment for free!**
 - This is cool because we never explicitly trained an alignment system
 - The network just learned alignment by itself



	Les	pauvres	sont	démunis
The	■			
poor		■		
don't			■	■
have			■	■
any			■	■
money			■	■